

Advanced Praktika SoSe 26 Notes

Igor Dimitrov

2026-04-21

Table of contents

Preface	3
I Notes	4
1 Project Setup	5
1.1 General repository structure	5
1.1.1 Experimental Drivers	6
1.2 HDNUM, CPFloat and GMP	7
1.3 Changing Top-level CMakeLists.txt	8
1.3.1 How to integrate <code>src/</code> later	9
1.3.2 A flexible version	10
1.3.3 Rule of thumb	11
2 Project Summary	12
2.1 Abstract	12
2.1.1 Problem	12
2.1.2 Algorithm	13
2.1.3 HDNUM & Own Implementation	13
2.1.4 Mixing Types	14
2.1.5 Methodology	14
2.1.6 What Should be Studied	15
2.1.7 Plan	16
3 Conversions	17
3.1 Floating Point Types	17
3.2 Type Conversions	19
3.2.1 Implementing Conversions - Naive Approach	19
3.2.2 Structure of FP and CPFloat Classes	24
3.2.3 Implementing Conversions - Correct Approach	25
3.3 Testing <code>hdnum_conversions.hpp</code>	33
3.3.1 Scalar Conversion Tests	34
3.3.2 Lossy Conversion Tests	34
3.3.3 Lossless Conversion Tests	34
3.3.4 Vector Conversion Tests	35

3.3.5	Matrix Conversion Tests	35
3.3.6	Representative Pair Types	36
4	Implementation of the Mixed IR Algorithm	37
4.1	Review of LU Decomposition	37
4.1.1	Partial Pivoting	37
4.1.2	Full Pivoting	37
4.1.3	Implementation in HDNum	38
4.2	The Mixed IR Algorithm	39
4.3	Unit Roundoff u	45
4.3.1	unit roundoff traits	46
5	Implementing the Experiments & Robustness Improvements	49
5.1	Temporary Matrix Generator	50
5.2	Matrix Generator Supervisor Version	50
5.2.1	Nonsymmetric Generator	51
5.3	Configurable test-problem Construction	52
5.3.1	Three Right-hand-side Modes	52
5.3.2	Test-problem Options	53
5.3.3	Linear-system Representation	54
5.3.4	Shared <code>complete_problem</code> function	54
5.3.5	High-precision reference solve	55
5.3.6	Matrix-generator interfaces	56
5.3.7	Usage Examples	56
5.4	Error Metrics Helper	58
5.5	First Convergence Test	58
5.6	Robustness Improvements to <code>mixed_ir</code>	60
5.6.1	1. FP16–FP64–FP128: overflow during input conversion	60
5.6.2	2. FP32–FP64–FP128: divergence of iterative refinement	61
5.6.3	Source-Code Changes	62
5.6.4	Result	66
5.6.5	FP16 residual underflow and stagnation problem	66
5.7	Plotting Convergence Experiments	75
5.7.1	Expected CSV format	76
5.7.2	Generated plots	76
5.7.3	Logarithmic plots and zero values	77
5.7.4	Command-line interface	77
5.7.5	Default behavior	78
5.7.6	Output tags	78
5.7.7	Descriptive plot labels	79
5.7.8	Matrix-family examples	79
5.7.9	Encoding the solution or RHS mode	80
5.7.10	Examples for the three modes	81

5.7.11	What the script detects automatically	82
5.7.12	Recording metadata inside the CSV	82
5.7.13	Later comparison plots	83
6	Mixed-Precision Iterative Refinement: Experiment Plan	84
6.1	Overall experiment plan	84
6.2	Group A: convergence histories	85
6.2.1	Representative precision triples	85
6.2.2	Condition numbers	85
6.2.3	Plots	85
6.3	Group B: general condition-number sweeps	86
6.3.1	Seed policy	86
6.3.2	Recorded quantities	86
6.3.3	Plots	87
6.4	Group C: single-double-double versus single-double-quad	87
6.4.1	Precision configurations	87
6.4.2	Theoretical expectation	88
6.4.3	Condition-number sweep	89
6.4.4	Required plots	89
6.4.5	Measurement details	90
6.5	Group D: comparison with a direct solve	90
6.5.1	Plots	91
6.6	Group E: FP16 robustness extension	91
6.6.1	E1. Controlled scaling comparison	91
6.6.2	Principal plots	92
6.6.3	Diagnostic plots	92
6.6.4	E2. Broader scaling validation	92
6.6.5	E3. FP16 overflow handling	93
6.7	Controlling the experiment space	93
6.8	Common metadata design	94
6.8.1	Experiment identity	94
6.8.2	Test-problem options	95
6.8.3	Algorithm options	95
6.8.4	Results	95
6.8.5	CSV representation	96
6.9	Automatic filenames	96
6.10	Output-directory organization	96
6.11	Recommended implementation order	97
6.11.1	Implementation Plan	98
6.11.2	1. Create the branch	98
6.11.3	2. Freeze the common schema first	98
6.11.4	3. Create <code>experiment_io.hpp</code>	99
6.11.5	4. Use the residual-scaling experiment as the pilot	101

6.11.6	5. Verify the pilot carefully	102
6.11.7	6. Keep the first milestone narrow	103
6.11.8	Condition Grids	103
6.11.9	Further Details	105
6.11.10	One template, several precision triples	106
6.11.11	Condition-number selection	108
6.11.12	Experiment settings	109
6.11.13	CSV organization	110
6.11.14	Plot organization	110
6.11.15	CMake organization	110
6.11.16	How to run	111
6.11.17	Legends Codes	112
7	Experimental Framework	113
7.1	Where the current design stands	114
7.2	What a typed experiment record would be	115
7.3	How the automatic CSV schema would work	117
7.4	Recommended record types	119
7.5	What configuration files would provide	119
7.5.1	One complication: precisions are compile-time types	121
7.6	Automatic precision-name traits	122
7.7	Research framework versus general-purpose library	123
7.7.1	1. Establish a stable numerical API	123
7.7.2	2. Separate numerical execution from observation	123
7.7.3	3. Abstract the factorization and matrix backend	124
7.7.4	4. Separate public headers and compiled utilities	124
7.7.5	5. Add systematic tests	125
7.7.6	6. Version external contracts	125
7.7.7	7. Provide packaging and documentation	125
7.8	A practical future roadmap	126
II	Report	127
8	Mixed-Precision Iterative Refinement	128
9	Introduction	129
10	Three-precision iterative refinement	130
10.1	Algorithm	130
10.2	Precision roles and theoretical expectations	130
11	Experimental setup	132
11.1	Configuration and common settings	132

11.2	Test problems	133
11.3	Precision formats	133
11.4	Error measures	134
11.5	Termination criteria and statuses	134
11.6	Experimental framework and reproducibility	134
12	Experimental design	135
13	Results and discussion	136
13.1	Convergence histories	136
13.1.1	Interpretation	136
13.2	Condition-number sweep	139
13.2.1	Interpretation	139
13.3	Residual-precision comparison	141
13.3.1	Interpretation	142
13.4	Direct-solve comparison	143
13.4.1	Interpretation	144
13.5	Low-precision limitations and residual scaling	145
13.5.1	Interpretation	147
13.6	Conclusion	148
	References	149
	Appendices	150
A	GDB usage Notes	150
A.1	Basic GDB Commands	150

Preface

This is a Quarto book.

To learn more about Quarto books visit <https://quarto.org/docs/books>.

Part I

Notes

1 Project Setup

1.1 General repository structure

The whole project is managed inside a Quarto book repository. The Quarto project contains notes, theory summaries, the final report, and the implementation code.

The planned directory structure is:

```
project-root/  
  index.qmd  
  report/  
  notes/  
  code/  
    CMakeLists.txt  
    external/  
      hdnum/          # git submodule  
        cpfloat/      # cloned inside hdnum  
    include/  
      mixed_ir.hpp    # algorithm  
      hdnum_conversions.hpp # convert()  
      error_metrics.hpp # forward/backward error helpers, later  
    experiments/  
      exp_convergence.cc  
      exp_condition_sweep.cc  
      exp_residual_precision.cc  
    examples/  
      hdnum_sandbox.cc  
    tests/  
      test_conversion.cc  
      test_mixed_ir_sanity.cc  
    scripts/  
      run_all_experiments.py  
      plot_results.py  
    results/  
      raw/  
      plots/
```

Purpose of the main sub-directories:

external/	external dependencies, especially HDNUM
include/	own header-only algorithm and helper code
src/	optional implementation files, if needed later
examples/	small sandbox/demo programs for learning and testing
experiments/	reproducible experimental drivers for report data
tests/	correctness and sanity checks
scripts/	automation and plotting scripts
results/raw/	generated CSV or data files
results/plots/	generated figures for the report

1.1.1 Experimental Drivers

They will go in:

```
code/  
  experiments/  
    exp_convergence.cc  
    exp_condition_sweep.cc  
    exp_residual_precision.cc
```

They will be written in c++ because they need to instantiate HDNUM types like FP16, bfloat16, FP64, FP128, FP256, run LU, perform refinement, and compute errors.

But the overall experiment workflow will be split:

```
C++ drivers:  
  run numerical experiments  
  write CSV files  
  
Python / shell scripts:  
  run many driver configurations  
  collect results  
  make plots
```

like:

```
code/
  experiments/
    exp_convergence.cc
    exp_condition_sweep.cc
    exp_residual_precision.cc
  scripts/
    run_all_experiments.py
    plot_results.py
  results/
    raw/
    plots/
```

The cpp experiments can be written so that they can be run like:

```
./build/exp_condition_sweep --uf FP16 --u FP64 --ur FP128 --n 100 --matrix random_orthogonal
./build/exp_condition_sweep --uf bfloat16 --u FP64 --ur FP128 --n 100 --matrix random_orthogonal
./build/exp_condition_sweep --uf FP32 --u FP64 --ur FP128 --n 100 --matrix random_orthogonal
```

They a python or shell script can run all of those automatically.

Precision sweeps should vary precision, condition number, and matrix type, and experiments should be scripted so rerunning them is convenient. In practice, that means your experiment system should let you rerun a whole table of experiments with one command, rather than manually recompiling/editing files.

1.2 HDNUM, CPFloat and GMP

HDNUM is included as a git submodule:

```
git submodule add https://parcomp-git.iwr.uni-heidelberg.de/Teaching/hdnum.git code/external/hdnum
git submodule update --init --recursive
```

CPFloat is cloned and built inside the HDNUM top-level directory, matching HDNUM's expected layout:

```
cd code/external/hdnum
git clone https://github.com/north-numerical-computing/cpfloat.git
cd cpfloat
make lib
```

GMP is installed system-wide:

```
sudo apt install libgmp-dev
```

Because CPFloat sits inside the HDNUM submodule but is not part of HDNUM itself, untracked files are hidden locally only for that submodule:

```
cd code/external/hdnum
git config status.showUntrackedFiles no
```

and the parent repository ignores untracked content specifically inside the HDNUM submodule:

```
git config submodule.code/external/hdnum.ignore untracked
```

1.3 Changing Top-level CMakeLists.txt

Currently the implementations are header-only and are contained in the folder include. Later if source files that not only header are added e.g. to a src/ folder it has to be accounted for:

Right now, this part:

```
add_library(mixed_precision_core INTERFACE)
```

is only an INTERFACE target. That means it does not compile any .cc files. It only carries usage requirements:

```
include_directories
compile_definitions
linked_libraries
compile_features
```

So it works well for a header-only project, but it does not yet build files from src/.

But if later you add:

```
src/csv_writer.cc
src/matrix_generators.cc
src/experiment_io.cc
```

then those files will not automatically be compiled unless we update CMake.

1.3.1 How to integrate src/ later

The usual clean solution is to change from a purely INTERFACE core target to a real library target.

For example:

```
add_library(mixed_precision_core
    src/csv_writer.cc
    src/matrix_generators.cc
    src/experiment_io.cc
)
```

Then keep the same include directories and dependencies:

```
target_include_directories(mixed_precision_core
    PUBLIC
        "${CMAKE_CURRENT_SOURCE_DIR}/include"
        "${HDNUM_DIR}"
        "${CPFLOAT_INCLUDE_DIR}"
)

target_compile_definitions(mixed_precision_core
    PUBLIC
        HDNUM_HAS_GMP=1
        HDNUM_HAS_CPFLOAT=1
)

target_link_libraries(mixed_precision_core
    PUBLIC
        "${CPFLOAT_LIBRARY}"
        gmpxx
        gmp
)

target_compile_features(mixed_precision_core
    PUBLIC
        cxx_std_17
)
```

Then all your examples, tests, and experiments can continue doing:

```
target_link_libraries(my_target
    PRIVATE
        mixed_precision_core
        mixed_precision_warnings
)
```

and they will automatically get both:

```
the headers from include/
the compiled implementation files from src/
```

1.3.2 A flexible version

If you do not want to manually list every source file, you can use globbing:

```
file(GLOB project_sources CONFIGURE_DEPENDS
    "${CMAKE_CURRENT_SOURCE_DIR}/src/*.cc"
    "${CMAKE_CURRENT_SOURCE_DIR}/src/*.cpp"
)

add_library(mixed_precision_core
    ${project_sources}
)
```

But there is one small issue: `add_library()` with an empty source list can be awkward in some CMake versions/workflows.

So for now, while `src/` is empty, I would keep the current `INTERFACE` version.

Later, when you add the first real `.cc` file, change:

```
add_library(mixed_precision_core INTERFACE)
```

to:

```
file(GLOB project_sources CONFIGURE_DEPENDS
    "${CMAKE_CURRENT_SOURCE_DIR}/src/*.cc"
    "${CMAKE_CURRENT_SOURCE_DIR}/src/*.cpp"
)

add_library(mixed_precision_core
```

```
    ${project_sources}  
)
```

and replace INTERFACE with PUBLIC in the relevant `target_*` commands.

1.3.3 Rule of thumb

Use this now:

```
add_library(mixed_precision_core INTERFACE)
```

while the project is header-only.

Use this later:

```
add_library(mixed_precision_core ${project_sources})
```

once `src/` contains real compiled support code.

2 Project Summary

2.1 Abstract

This *Fortgeschrittenenpraktikum* project investigates mixed precision iterative refinement for solving linear systems

$$Ax = b.$$

The goal is to implement a three-precision LU-based method: the LU factorization and correction solves use low precision u_f , the solution updates use working precision u , and the residuals are computed in higher precision u_r . Starting from a low-precision LU solve, the algorithm repeatedly computes $r_i = b - Ax_i$, solves for a correction d_i , and updates x_i until $|d_i|/|x_i| < u$ or a maximum iteration count is reached.

The focus is empirical accuracy, not runtime, since low precisions such as fp8, fp16, and bfloat16 are simulated via CPFloat. The experiments will study convergence, forward/backward error, condition-number dependence, precision triples (u_f, u, u_r) residual precision, and comparison with a direct working-precision solve.

Summary:

- implement three-precision variant of **iterative refinement for linear systems**
- study its accuracy

2.1.1 Problem

- A linear system $Ax = b$. **iterative refinement** improves an approximate solution by repeatedly computing the residual and solving a correction equation.
 - LU factorization in low precision: u_f
 - **working arithmetic** in ‘middle’ precision: u
 - residual computation in high precision: u_r

i Note

Review the definition of **accuracy** of an algorithm

- precision triples (u_r, u, u_f) with $u_f \succ u \succ u_r$:
 - i.e. u_f low precisions like: `fp8`, `fp16`, `bfloat16`, come from `CPFloat`:
 - * `FP8`: `CPFloat<4, 4>`
 - * `bfloat16`: `CPFloat<8, 8>`
 - * `FP16`: `CPFloat<11, 5>`
 - u is probably the native `FP64` i.e. double precision

i Note

Study the `FP8`, `FP16` and `bfloat16` formats and how they relate to template parameters in `CPfloat<>`

2.1.2 Algorithm

Algorithm: Three-precision iterative refinement

1. Compute the LU factorization $PA = LU$ in precision u_f .
2. Solve $LU x_0 = P b$ in precision u_f ; cast x_0 to precision u .
3. While not converged:
 4. Compute $r_i = b - A x_i$ in precision u_r .
 5. Cast r_i to precision u .
 6. Solve $LU d_i = P r_i$ in precision u_f ; cast d_i to precision u .
 7. Update $x_{i+1} = x_i + d_i$ in precision u .

- convergence condition: $\frac{\|d_i\|}{\|x_i\|} < u$ where u is the unit roundoff of the working precision,
- or stop after a max number of iterations.

i Note

why is $\frac{\|d_i\|}{\|x_i\|} < u$ the convergence condition?

2.1.3 HDNUM & Own Implementation

relevant parts:

matrix and vector classes:

- `src/densematrix.hh`: `DenseMatrix<T>` with:
 - `mv`: matrix vector product
 - `mm`: matrix matrix product
 - `transpose()` etc
- `src/vector.hh`: `Vector<T>` with vector arithmetic operations (multiplication with scalar, addition) and norms
- LU decomposition in `src/lr.hh`:
 - `lr_fullpivot(A, p, q)`
 - `lr_partialpivot(A, p)`
 - triangular solves: `solveL(A, x, b)`, `solveR(A, x, b)`
 - `permute_forward` and `permute_backward`

2.1.4 Mixing Types

In a mixed precision algorithm different parts of the algorithm are instantiated / computed with different precisions, i.e. different types:

- LU factorization is done in lower precision u_f : E.g. `DenseMatrix<FP16>`
- Residual computation in higher precision u_r : E.g. `DenseMatrix<FP128>`

Therefore we must be able to convert from one type (class) like `DenseMatrix<FP16>` to another type (class) like `DenseMatrix<FP128>`. This conversion requires explicit construction or element-wise copying.

Managing type conversions between precisions is they key challenge. It is suggested to do this with a explicit template functions for matrices and vectors:

```
template<class T_out, class T_in>
void convert(Vector<T_out>& out, const Vector<T_in>& in)

template<class T_out, class T_in>
void convert(DenseMatrix<T_out>& out, const Vector<T_in>& in)
```

2.1.5 Methodology

- **reference solutions**: run a standard solver for $A \cdot x = b$ in FP256 and treat the result x_{ex} as exact.
- **error metrics**:
 - report errors using norms appropriate to your algorithm

- compute **all error measures** in a precision higher than the working precision of your algorithm, i.e. u
- otherwise you are measuring the precision of your measurement.

i Note

- What does **error** mean in this context? $\|x^* - x_{ex}\|_2$?
- What is a norm that is **appropriate** to LU, simply $\|\bullet\|_2$?
- Does Compute error measures in a precision higher than u simply mean that we compute $\|\cdot\|$ in something like FP128
 - what does “otherwise you are measuring the precision of your measurement” mean?

- experiments should be scripted so that re-running them with various precision, condition number and matrix type is convenient.

2.1.6 What Should be Studied

- **Convergence Histories:** for each precision combination (u_f, u, u_r) and several test matrices A_i with different condition numbers $k(A_i)$:
 - plot **forward error** as a function of i
 - plot **backward error** as a function of i

i Note

review the definitions of **forward error** and **backward error**

- **Summary Across Condition Numbers:** for a wide range of $k(A)$ for each precision combinations
 - record:
 - * the final achieved **forward error**
 - * number of iterations
 - plot these as a function of $k(A)$: theory predicts convergence requires roughly $k(A) \cdot u_f < 1$.
- **The Role u_r :** Fix (u_f, u) and vary u_r . When does $u_r > u$ make a measurable difference

i Note

- Normally $u_r < u$, i.e. the residual computation is done in a precision higher than the working precision.
- What happens when we lower the precision, when does it make a difference?

2.1.7 Plan

1. Implement `hdnum_conversions.hpp`.
2. Add conversion tests.
3. Implement a minimal `mixed_ir.hpp`.
4. First test with all precisions set to FP64.
5. Then test real mixed precision triples.
6. Add systematic experiments and plotting scripts.

3 Conversions

3.1 Floating Point Types

In this project there are three groups of number formats:

1. simulated low precision via CPFloat
2. native C++ hardware types `float` and `double`
3. high / arbitrary precision via GMP

The low precision `fp8`, `fp16`, `bfloat16` are simulated via CPFloat.

An overview of types defined in HDNUM:

HDNUM / project type	Provider	Parameters	Corresponds to in Higham–Mary	Unit roundoff, roughly
FP8	CPFloat	CPFloat<4,4>	custom 8-bit / “quarter precision”-like format	$2^{-4} \approx 6.25 \cdot 10^{-2}$
bfloat16	CPFloat	CPFloat<8,8>	Higham–Mary <code>bfloat16</code>	$2^{-8} \approx 3.91 \cdot 10^{-3}$
FP16	CPFloat	CPFloat<11,5>	IEEE half precision, Higham–Mary <code>fp16</code>	$2^{-11} \approx 4.88 \cdot 10^{-4}$
FP32	native C++ <code>float</code>	effectively (24,8)	IEEE single precision, Higham–Mary <code>fp32</code>	$2^{-24} \approx 5.96 \cdot 10^{-8}$
FP64	native C++ <code>double</code>	effectively (53,11)	IEEE double precision, Higham–Mary <code>fp64</code>	$2^{-53} \approx 1.11 \cdot 10^{-16}$
FP128	GMP	approx. 128-bit, FP<64>	software high precision / “quadruple-like” role, not necessarily IEEE <code>fp128</code>	depends on HDNUM wrapper
FP256	GMP	approx. 256-bit, FP<192>	multiprecision reference precision	depends on chosen GMP precision

HDNUM / project type	Provider	Parameters	Corresponds to in Higham–Mary	Unit roundoff, roughly
FP512, FP1024	GMP	approx. 512 / 1024-bit	multiprecision, beyond Higham–Mary’s standard table	depends on chosen GMP precision

As a reminder the three-precision LU-IR algorithm is:

1. Compute the LU factorization $PA = LU$ in precision u_f .
2. Solve $LU x_0 = P b$ in precision u_f ; cast x_0 to precision u .
3. While not converged:
 4. Compute $r_i = b - A x_i$ in precision u_r .
 5. Cast r_i to precision u .
 6. Solve $LU d_i = P r_i$ in precision u_f ; cast d_i to precision u .
 7. Update $x_{i+1} = x_i + d_i$ in precision u .

i.e.:

- 1) Compute LU in u_f
- 2) Solve the initial system in u_f
- 3) Compute residuals in u_r
- 4) Cast residuals to u
- 5) Solve correction equations using the low-precision LU factors in u_f
- 6) update in u

For the algorithm the intended mapping is:

Algorithm role	Meaning	Typical project choices
u_f	factorization precision; low precision used for LU	FP8, bfloat16, FP16, sometimes FP32
u	working precision; stores A, b, x_i and updates solution	usually FP32 or FP64
u_r	residual precision; computes $r_i = b - Ax_i$ accurately	FP64, FP128, maybe FP256 for experiments

A reasonable set of experiments could be:

Experiment	u_f	u	u_r
baseline sanity check	FP64	FP64	FP64
classical mixed precision	FP32	FP64	FP64
half/double refinement	FP16	FP64	FP64
half/double with higher residual	FP16	FP64	FP128
bfloat comparison	bfloat16	FP64	FP64
very low precision stress test	FP8	FP64	FP128

3.2 Type Conversions

Again, the precisions for the algorithm are:

- 1) u_f : factorization / solve precision (low)
- 2) u : working precision (medium)
- 3) u_r : residual precision (high)

LU is computed in u_f , the initial solution is cast to u , the residual is computed in u_r and cast back to u , the correction solve uses the low-precision LU factors, and the correction is cast to u before updating.

So the practically necessary conversions are:

Conversion		
$u \rightarrow u_f$		create low-precision copies of A , b , or residuals for LU / triangular solves.
$u_f \rightarrow u$		bring x_0 and correction d_i back to working precision.
$u \rightarrow u_r$		compute $r_i = b - Ax_i$ in high precision.
$u_r \rightarrow u$		The residual is cast back before solving the correction equation.
$u_f \leftrightarrow u_r$	not strictly needed	You normally convert through the working representation, or independently convert from the original A, b, x .

3.2.1 Implementing Conversions - Naive Approach

HDNum implements wrappers `FP` and `CPFloat` for the high-precision GMP and the low precision `CPFloat` types in the source files `highprec_gmp.hh` and `lowprec_cpfloat.hh`.

We are asked not to modify HDNum source files, instead implement explicit conversion functions:

```

template<class T_out, class T_in>
void convert(Vector<T_out>& out, const Vector<T_in>& in);

template<class T_out, class T_in>
void convert(DenseMatrix<T_out>& out, const DenseMatrix<T_in>& in);

```

The approach is to assign to `out` element-wise in loops, where the values constructed from the corresponding elements of `in`. A naive approach is to do:

```

#pragma once

#include <cstdlib>
#include "hdnum.hh"

namespace mpir {

// Vector conversion: resizes output if needed.
template<class T_out, class T_in>
void convert(hdnum::Vector<T_out>& out,
             const hdnum::Vector<T_in>& in)
{
    if (out.size() != in.size()) {
        out.resize(in.size());
    }

    for (std::size_t i = 0; i < in.size(); ++i) {
        out[i] = T_out(in[i]);
    }
}

// Convenience wrapper: allocates a new vector.
template<class T_out, class T_in>
hdnum::Vector<T_out> convert_vector(const hdnum::Vector<T_in>& in)
{
    hdnum::Vector<T_out> out(in.size());
    convert(out, in);
    return out;
}

// Matrix conversion: output matrix must already have correct shape.
template<class T_out, class T_in>

```

```

void convert(hdnum::DenseMatrix<T_out>& out,
             const hdnum::DenseMatrix<T_in>& in)
{
    if (out.rowsize() != in.rowsize() || out.colsize() != in.colsize()) {
        throw std::invalid_argument(
            "mpir::convert(DenseMatrix): output matrix has wrong size"
        );
    }

    for (std::size_t i = 0; i < in.rowsize(); ++i) {
        for (std::size_t j = 0; j < in.colsize(); ++j) {
            out[i][j] = T_out(in[i][j]);
        }
    }
}

// Convenience wrapper: allocates a new matrix.
template<class T_out, class T_in>
hdnum::DenseMatrix<T_out> convert_matrix(const hdnum::DenseMatrix<T_in>& in)
{
    hdnum::DenseMatrix<T_out> out(in.rowsize(), in.colsize());
    convert(out, in);
    return out;
}

```

Where the type conversion is done via the assignment:

```
out[i] = T_out(in[i])
```

for each element.

But this doesn't work for all pairs of `T_in`, `T_out` because not all the necessary constructors or conversion operators are implemented in the classes `CPFloat` and `FP`

Which Conversions Work which Don't ?

FP provides the constructors:

- `FP()`
- `FP(const double &)`
- `FP(const float &)`
- `FP(const int &)`

- `FP(const mpf_class &)`
- `FP<mm>(const FP<mm>&)`

So the conversion:

- `double ==> GMP`

works

`CPFloat` provides the constructors:

- `CPFloat()`
- `CPFloat(const double &)`
- `CPFloat<mm, ee>(const double &)`

So:

- `double ==> CPFloat`

works

`CPFloat` also provides the conversion operator `operator double() const`. Therefore

- `CPFloat ==> double`

works as well.

Because `CPFloat` can be converted to `double` and `GMP` can be constructed from a `double`:

- `CPFloat ==> GMP`

works too, by the implicit conversion of `CPFloat` to a `double`

But since the `FP` class doesn't provide a `double()` operator the conversion

- `GMP ==> double`

doesn't work.

Also `CPFloat` doesn't implement a constructor that accepts a `GMP` type, (or also because `FP` doesn't provide a `double()` operator) the conversion:

- `GMP ==> CPFloat`

doesn't work.

We summarize this with the following diagram and table:

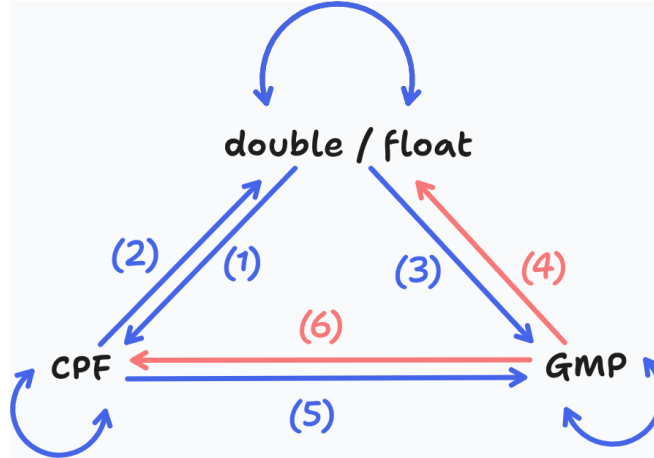


Figure 3.1: conversion triangle

Table 3.5: conversion table

Nr	Conversion	Works	
		?	Why ?
(1)	<code>double</code> $\Rightarrow$ <code>CPFloat</code>	Yes	<code>CPFloat(const double &)</code>
(2)	<code>CPFloat</code> $\Rightarrow$ <code>double</code>	Yes	<code>CPFloat::double()</code>
(3)	<code>double</code> $\Rightarrow$ <code>GMP</code>	Yes	<code>FP(const double &)</code>
(4)	<code>GMP</code> $\Rightarrow$ <code>double</code>	No	No <code>FP::double()</code>
(5)	<code>CPFloat</code> $\Rightarrow$ <code>GMP</code>	Yes	<code>CPFloat</code> is converted to <code>double</code> implicitly
(6)	<code>GMP</code> $\Rightarrow$ <code>CPFloat</code>	No	No <code>FP(const CPFloat &)</code> or no <code>FP::double()</code>

So if the conversion operator `FP::double()` would be implemented both (4) and (6) would work. (6) would work too because `GMP` type would be implicitly converted to a `double` and a `CPFloat` would be constructed. But more explicitly a constructor `CPFloat(const FP &)` can be implemented too.

But since we are initially not allowed to modify HDNUM source code we take a different approach.

3.2.2 Structure of FP and CPFloat Classes

FP	CPFloat
-mpf_class number	-double number
+FP() +FP(const double &) +FP(const mpf_class &) +FP(...) +getNumber() : mpf_class	+CPFloat() +CPFloat(const double &) +CPFloat(...) +operator double() +getNumber() : double

- FP:
 - FP has a single private member variable `number` that is an GMP type, i.e. `mpf_class`.
 - it can be accessed with the public getter `const getNumber() : mpf_class`
- CPFloat:
 - CPFloat has a single private member variable `number` of type `double`.
 - it can be accessed with the public getter `const getNumber() : double`

Note

There's a mistake in HDNUM `highprec_gmp.hh` source file. The member variable `number` is documented as:

Listing 3.1 `highprec_gmp.hh`

```
private:  
    // stores a double to represent the low precision number  
    mpf_class number;
```

The comment `// stores a double to represent the low precision number` is wrong, and is probably copied over by mistake from `lowprec_cpfloat.hh`:

Listing 3.2 lowprec_cpfloat.hh

```
private:
    // stores a double to represent the low precision number
    double number;
```

Instead it should be something like:

```
// stores an mpf_class to represent the high precision GMP number
```

With the above interface, given a GMP number, the double representation of the number can be obtained with:

Listing 3.3 get_d() method

```
hdnum::FP128 fp128_number(1.2345);
auto d = fp128_number.getNumber().get_d(); //d is double
```

Where we use the `.get_d()` method of `mpf_class` to get the `double` representation of the GMP number.

3.2.3 Implementing Conversions - Correct Approach

The main issue is that

```
out[i] = T_out(in[i]);
```

does not work uniformly for all type pairs. We replace this direct construction step with:

```
out[i] = scalar_convert<T_out>(in[i]);
```

This selects the appropriate conversion path at compile time, depending on the input and output types using standard library type traits and `if constexpr`

As summarized in Figure 3.1 and Table 3.5, the only conversions that do not work by direct construction are:

1. FP $\Rightarrow$ double,
2. FP $\Rightarrow$ CPFloat

All other relevant cases can be handled by direct construction of the form:

```
T_out(x) // where x has type T_in
```

In C++, we can check at compile time whether an object of type `T_out` can be constructed from an argument type `const T_in&` using the standard library type trait:

```
std::is_constructible_v<T_out, const T_in&>
```

This expression evaluates to `true` if such a construction is valid, and to `false` otherwise.

It can be verified that this indeed evaluates to `false` only for the conversion pairs:

- `T_out == double, T_in == FP`
- `T_out == CPFloat, T_in == FP`

Then, the initial part of `scalar_cast` can be written as:

```
template<class T_out, class T_in>
T_out scalar_cast(const T_in& x)
{
    if constexpr (std::is_constructible_v<T_out, const T_in&>) {
        return T_out(x); // we use the direct construction
    }
    else { //remaining cases: FP => double || FP => CPFloat
        ...
    }
}
```

Logically, we know that the only possible cases in the `else` branch are:

- `T_out == double, T_in == FP`
- `T_out == CPFloat, T_in == FP`

In both cases `T_in` is an FP type. as we've seen in Listing 3.3 the `double` representation of an FP number `x` can be extracted with:

```
x.getNumber().get_d()
```

Since both `CPFloat` and `double` can be constructed from a `double`, we could simply write:

```

template<class T_out, class T_in>
T_out scalar_cast(const T_in& x)
{
    if constexpr (std::is_constructible_v<T_out, const T_in&>) {
        return T_out(x); // we use the direct construction
    }
    else { //remaining cases: FP => double || FP => CPFloat
        return T_out(x.getNumber().get_d());
    }
}

```

However, this version is not robust enough because the `else` branch relies on our prior reasoning that only the two exceptional cases can reach it. The code itself does not express this assumption. If `scalar_cast` is later instantiated with another unsupported conversion pair, the compiler will still try to compile

```
x.getNumber().get_d()
```

even if `T_in` is not an FP type. This should lead to implementation-dependent and potentially confusing error message.

Therefore, it is better to make the remaining cases explicit using another `if constexpr` condition:

```

template<class T_out, class T_in>
T_out scalar_cast(const T_in& x)
{
    if constexpr (std::is_constructible_v<T_out, const T_in&>) {
        return T_out(x); // direct construction works
    }
    else if constexpr (
        is_hdnum_fp_v<T_in> &&
        std::is_constructible_v<T_out, double>
    ) {
        return T_out(x.getNumber().get_d());
    }
    else {
        static_assert(always_false<T_out, T_in>::value,
            "unsupported scalar conversion");
    }
}

```

Here, the helper trait `is_hdnum_fp_v<T>` detects whether a type `T` is one of HDNUM's GMP-based high precision number types `hdnum::FP<m>`. This is a compile-time Boolean value, and analogous to the standard library traits such as `std::is_constructible_v`.

The implementation, given below, follows a common template metaprogramming partial specialization pattern:

```
// Detect HDNUM's GMP wrapper type hdnum::FP<m>
template<class T>
struct is_hdnum_fp_impl : std::false_type {};

template<int m>
struct is_hdnum_fp_impl<hdnum::FP<m>> : std::true_type {};

template<class T>
inline constexpr bool is_hdnum_fp_v =
    is_hdnum_fp_impl<
        std::remove_cv_t<std::remove_reference_t<T>>
    >::value;
```

1) The first part

```
template<class T>
struct is_hdnum_fp_impl : std::false_type {};
```

is the default case and says:

for an arbitrary type `T`, it is false

So:

```
is_hdnum_fp_impl<double>::value      // false
is_hdnum_fp_impl<float>::value        // false
is_hdnum_fp_impl<hdnum::FP16>::value // false, because CPFloat, not GMP FP
```

2) The second part

```
template<int m>
struct is_hdnum_fp_impl<hdnum::FP<m>> : std::true_type {};
```

is the specialization case, and says:

if the type has the form `hdnum::FP<m>`, it is true.

So:

```
is_hdnum_fp_impl<hdnum::FP128>::value // true
is_hdnum_fp_impl<hdnum::FP256>::value // true
```

3) The last part

```
template<class T>
inline constexpr bool is_hdnum_fp_v =
    is_hdnum_fp_impl<
        std::remove_cv_t<std::remove_reference_t<T>>
    >::value;
```

defines a compile-time boolean variable template. For example:

```
is_hdnum_fp_v<double> // false
is_hdnum_fp_v<hdnum::FP128> // true
```

It is analogous to standard library helpers such as:

```
std::is_arithmetic_v<T>
std::is_constructible_v<T, Args...>
std::is_convertible_v<From, To>
```

and allows to write

```
is_hdnum_fp_v<T>
```

instead of

```
is_hdnum_fp_impl<T>::value
```

The keyword `inline` is needed for variable that are defined in header files.

The use of:

```
std::remove_reference_t<T>
std::remove_cv_t<T>
```

makes the trait insensitive to references and `const` / `volatile` qualifiers. For example this way all of the following are treated as the same:

```
hdnum::FP<64>
const hdnum::FP<64>
hdnum::FP<64>&
const hdnum::FP<64>&
```

Without removing references and cv-qualifiers, the specialization would not match types such as `hdnum::FP<m>&`

The final branch of `scalar_cast`

```
else{
    static_assert(always_false<T_out, T_in>::value,
        "unsupported scalar conversion");
}
```

This branch is reached only if neither of the previous compile-time conditions applies:

```
std::is_constructible_v<T_out, const T_in&>
```

is `false`, so direct construction does not work, and

```
is_hdnum_fp_v<T_in> && std::is_constructible_v<T_out, double>
```

is also `false`, so the special `FP => double / FP => CPFloat` conversion path doesn't apply either.

Therefore, if the compiler reaches this final 'else' branch, the requested scalar conversion is unsupported. In that case, we want compilation to fail with a clear error message.

The helper trait is defined as:

```
template<class...>
struct always_false : std::false_type {};
```

This defines a type trait that is always `false`, regardless of the template arguments. For example:

```
always_false<int>::value           // false
always_false<double, int>::value   // false
always_false<hdnum::FP<64>, double>::value // false
```

The `class...` syntax denotes a template parameter pack and means that `always_false` can accept any number of template type arguments. In our case we use two arguments:

```
always_false<T_out, T_in>::value
```

It is important that `always_false<T_out, T_in>::value` depends on template parameters `T_out` and `T_in`, as it lets us create a compile-time `false` value that is evaluated when the templates are instantiated and not when the function is parsed. Omitting and writing simply `static_assert(false, ...)` would cause the assertion to be checked when the function template is parsed, and not only when the template is instantiated and the final branch is actually selected for the concrete instantiation.

scalar_cast Final Version

```
#pragma once

#include <cstdint>
#include <stdexcept>
#include <type_traits>

#include "hnum.hh"

namespace mpir {

// Helper for dependent static_assert(false)
template<class...>
struct always_false : std::false_type {};

// Detect HDNUM's GMP wrapper type hnum::FP<m>
template<class T>
struct is_hnum_fp_impl : std::false_type {};

template<int m>
struct is_hnum_fp_impl<hnum::FP<m>> : std::true_type {};

template<class T>
inline constexpr bool is_hnum_fp_v =
    is_hnum_fp_impl<
        std::remove_cv_t<std::remove_reference_t<T>>
    >::value;

// Scalar conversion
template<class T_out, class T_in>
T_out scalar_cast(const T_in& x)
{
    if constexpr (std::is_constructible_v<T_out, const T_in&>) {
        return T_out(x);
    }
    else if constexpr (
        is_hnum_fp_v<T_in> &&
        std::is_constructible_v<T_out, double>
    ) {

```

```

        return T_out(x.getNumber().get_d());
    }
    else {
        static_assert(always_false<T_out, T_in>::value,
            "mpir::scalar_cast: unsupported scalar conversion");
    }
}

// Vector conversion
template<class T_out, class T_in>
void convert(hdnum::Vector<T_out>& out,
             const hdnum::Vector<T_in>& in)
{
    if (out.size() != in.size()) {
        throw std::invalid_argument(
            "mpir::convert(Vector): output vector has wrong size"
        );
    }

    for (std::size_t i = 0; i < in.size(); ++i) {
        out[i] = scalar_cast<T_out>(in[i]);
    }
}

// DenseMatrix conversion
template<class T_out, class T_in>
void convert(hdnum::DenseMatrix<T_out>& out,
             const hdnum::DenseMatrix<T_in>& in)
{
    if (out.rowsize() != in.rowsize() || out.colsize() != in.colsize()) {
        throw std::invalid_argument(
            "mpir::convert(DenseMatrix): output matrix has wrong size"
        );
    }

    for (std::size_t i = 0; i < in.rowsize(); ++i) {
        for (std::size_t j = 0; j < in.colsize(); ++j) {
            out[i][j] = scalar_cast<T_out>(in[i][j]);
        }
    }
}

```

```

    }
}

// convenience function
template<class T_out, class T_in>
hdnum::DenseMatrix<T_out> convert_matrix(const hdnum::DenseMatrix<T_in>& in)
{
    hdnum::DenseMatrix<T_out> out(in.rowsize(), in.colsize());
    convert(out, in);
    return out;
}

// convenience function
template<class T_out, class T_in>
hdnum::Vector<T_out> convert_vector(const hdnum::Vector<T_in>& in)
{
    hdnum::Vector<T_out> out(in.size());
    convert(out, in);
    return out;
}

} // namespace mpir

```

3.3 Testing `hdnum_conversions.hpp`

The file `test_conversion.cc` tests the conversion infrastructure implemented in `hdnum_conversions.hpp`. Its goals are to test that:

1. `scalar_cast<T_out>(x)` compiles for representative type.
2. `scalar_cast<T_out>(x)` returns the requested output type.
3. lossy low-precision conversions visibly lose information.
4. `convert(Vector)` applies `scalar_cast` element-wise correctly.
5. `convert(DenseMatrix)` applies `scalar_cast` elementwise correctly.
6. Vector and Matrix sizes are handled correctly.
7. Wrong-sized output containers throw exceptions
8. The convenience functions `convert_vector` and `convert_matrix` allocate correctly.

3.3.1 Scalar Conversion Tests

The scalar tests check that

```
mpir::scalar_cast<T_out>(x)
```

can be instantiated for representative input/output type pairs. For each pair, several sample values are tested:

```
0.0  
1.0  
-2.5  
1.0 / 3.0  
1.23456789012345
```

The result is checked at compile time with

```
static_assert(std::is_same_v<decltype(y), T_out>);
```

To verify that `scalar_cast<T_out>(x)` returns the requested output type `T_out`.

Both the input and the converted value are transformed to their approximate `double` representation with `approx_as_double` in order to compare the values with various `assert` and inspect the outputs.

3.3.2 Lossy Conversion Tests

It is verified that conversion from `double` to `CPFloat` indeed loses precision using a value such as 1.23456789012345.

This is only a useful sanity check that the conversion is really taking place.

3.3.3 Lossless Conversion Tests

It is also verified that conversion from a lower precision to a higher precision do not indeed lose information.

3.3.4 Vector Conversion Tests

Vector tests check the function

```
mpir::convert(hdnum::Vector<T_out>& out,  
              const hdnum::Vector<T_in>& in)
```

for a representative vector of length 5 and checks that each component of the output agrees with applying `scalar_cast<T_out>` to the corresponding input component.

The comparison is performed approximately by converting both values to `double` and using a small relative tolerance with the function

```
inline bool close(double a, double b, double atol = 0.0, double rtol = 1e-12)  
{  
    const double scale = std::max(std::abs(a), std::abs(b));  
    return std::abs(a - b) <= atol + rtol * scale;  
}
```

This function says whether the two double values are close enough to be considered equal. The tolerance is scaled by the max value of the inputs.

The test also includes the edge cases:

- vector size 0
- vector size 1

Additionally wrong-sized output vector is checked to verify whether an exception is thrown.

Finally the convenience function

```
mpir::convert_vector<T_out>(in)
```

is tested, by verifying the resulting type and converted values.

3.3.5 Matrix Conversion Tests

The matrix tests check the function

```
mpir::convert(hdnum::DenseMatrix<T_out>& out,  
              const hdnum::DenseMatrix<T_in>& in)
```

on a representative 5 x 5 matrix, by looping over the output values and comparing them against the result of applying `scalar_cast` to corresponding input entries.

The test includes the edge case of 1 x 1 matrix and a wrong-sized test.

The convenience function `mpir::convert_matrix<T_out>(in)` is also tested.

3.3.6 Representative Pair Types

1. Native $\leftrightarrow$ Native:
 - FP32 $\leftrightarrow$ FP64
2. Working precision $u \rightarrow$ low precision u_f :
 - FP64 $\rightarrow$ FP16 / bfloat16 / FP8
3. Low Precision $u_f \rightarrow$ working precision u :
 - FP16 / bfloat16 / FP8 $\rightarrow$ FP64
4. Working precision $u \rightarrow$ residual precision u_r
 - FP64 $\rightarrow$ FP128 / FP256
5. Residual precision $u_r \rightarrow$ working precision u
 - FP128 / FP256 $\rightarrow$ FP64
6. Low precision $u_f \leftrightarrow$ high residual/reference precision u_r
 - FP128 / FP256 $\leftrightarrow$ FP16 / bfloat16 / FP8
7. High precision $\leftrightarrow$ high precision
 - FP128 $\leftrightarrow$ FP256

Covering the main conversion direction required later by the algorithm:

- $u \rightarrow u_f$
- $u_f \rightarrow u$
- $u \rightarrow u_r$
- $u_r \rightarrow u$

The tests also include low to high-precision conversions not required directly by the algorithm but useful nevertheless.

4 Implementation of the Mixed IR Algorithm

4.1 Review of LU Decomposition

4.1.1 Partial Pivoting

In partial pivoting only rows are possibly permuted and the permutations are ‘stored’ in a matrix P . So we compute the decomposition:

$$LU = PA$$

This can be used to solve the matrix equation $Ax = b$ as follows:

$$\begin{aligned} Ax &= b \\ \Leftrightarrow (PA)x &= Pb \\ \Leftrightarrow L(Ux) &= Pb & (LU = PA, \text{ assoc}) \\ \Rightarrow Ly &= Pb & (y := Ux) \\ \text{solve for } y &\text{ using forward subst} \\ \text{solve } Ux = y &\text{ for } x \text{ using backward subst} \end{aligned}$$

4.1.2 Full Pivoting

In full pivoting columns can be permuted in addition to the row permutation, improving backward stability of the algorithm, s.t. the following equation holds:

$$PAQ = LU$$

Here P , Q are the row and column permutation matrices. This decomposition can be used to solve the equation $Ax = b$ as follows:

$$\begin{aligned}
& Ax = b \\
& \Leftrightarrow PA(QQ^{-1})x = b \\
& \Leftrightarrow (PAQ)Q^{-1}x = b \\
& \Leftrightarrow L(UQ^{-1})x = b & (LU = PAQ) \\
& \Rightarrow Lz = b & (UQ^{-1} := z)
\end{aligned}$$

Solve for z using forward subst.

We have:

$$\Rightarrow Uy = z \quad (Q^{-1}x := y)$$

Solve for y using backward subst.

Compute $x = Qy$

4.1.3 Implementation in HDNum

Solving $Ax = b$ using LU decomposition is implemented in HDnum as follows. Consider:

$$A = \begin{bmatrix} 2 & 1 & 7 \\ 8 & 8 & 33 \\ -4 & 10 & 4 \end{bmatrix}, \quad b = \begin{bmatrix} 15 \\ 73 \\ 12 \end{bmatrix}$$

We enter this as follows:

```

hdnum::Vector<hdnum::FP256> b(3);
b[0] = 15;
b[1] = 73;
b[2] = 12;

hdnum::DenseMatrix <hdnum::FP256> A(3, 3);
A[0][0] = 2;   A[0][1] = 1;   A[0][2] = 7;
A[1][0] = 8;   A[1][1] = 8;   A[1][2] = 33;
A[2][0] = -4;  A[2][1] = 10;  A[2][2] = 4;

```

We also need the vector x to hold the solution, and the vectors p and q to hold the row and column permutations:

```

hdnum::Vector<hdnum::FP256> x(3, 0.0); // holds the solution
hdnum::Vector<std::size_t> p(3);
hdnum::Vector<std::size_t> q(3);

```

Then we carry out the LU decomposition as follows:

```
hdnum::lr_fullpivot(A, p, q);
```

After this command A is modified and holds both L and U , while p and q hold the permutation information. We also have to permute b accordingly:

```
hdnum::permute_forward(p, b);
```

Then we solve the $Ly = b$ part of the equation (the forward substitution):

```
hdnum::solveL(A, b, b);
```

Note that we are overwriting b and storing the result directly in it to save space. Then, we finally complete the solution with:

```
hdnum::solveR(A, x, b);
```

Since we did full-pivoting, we should permute back the solution with:

```
hdnum::permute_backward(q, x);
```

Now x holds the solution.

4.2 The Mixed IR Algorithm

First we remember the mixed-ir algorithm:

1. Compute LU factorization $PAQ = LU$ in precision uf .
2. Solve LU $x_0 = P b$ in precision uf ;
Cast x_0 to precision u
3. While not converged:
 4. Compute $r_i = b - A x_i$ in precision ur (residual)
 5. cast r_i to precision u .
 6. Solve LU $d_i = P r_i$ in precision uf ;
Cast d_i to precision u
 7. update $x(i+1) = x_i + d_i$ in precision u

- Convergence condition $\frac{\|d_i\|}{\|x_i\|} < u$, where u is the unit roundoff of the **working precision**, or stop after a max number of iterations.

The c++ implementation of this algorithm is given as follows. First we define an options struct that holds the options that control max iterations and relative correction tolerance, i.e. the termination condition.

```
struct MixedIROptions {  
    std::size_t max_iterations = 10;  
    double rel_correction_tol = 0.0;  
};
```

The default value `rel_correction_tol = 0.0` causes the algorithm to try to use the default unit roundoff of the data type used for the working precision u , as we'll see later in the algorithm.

Next we define a struct to hold the result, here the comments provide the explanations:

```
template<class T_work>  
struct MixedIRResult {  
    // Final approximate solution, stored in the working precision T_work.  
    hdnum::Vector<T_work> x;  
  
    // Number of refinement iterations actually performed.  
    // This does not count the initial LU solve for x0.  
    std::size_t iterations = 0;  
  
    // True if the stopping criterion was reached before max_iterations.  
    // False means the algorithm stopped because the iteration limit was reached.  
    bool converged = false;  
  
    // Last computed relative correction norm ||d_i|| / ||x_i||.  
    // Useful for diagnostics, especially if converged == false.  
    double final_rel_correction = 0.0;  
  
    // History of relative correction norms, one entry per refinement iteration.  
    // Useful for convergence diagnostics and later plotting.  
    std::vector<double> rel_corrections;  
};
```

We define a function that returns u of the templated type T , if it's obtainable, or throws an error if not. (Mostly we will use `double` for u , for which it can be computed in compile time):

```

template<class T>
double default_unit_roundoff()
{
    static_assert(std::numeric_limits<T>::is_specialized,
        "default_unit_roundoff: no numeric_limits specialization for this type;"
        "please set_rel_correction_tol_explicitly");

    return 0.5 * static_cast<double>(std::numeric_limits<T>::epsilon());
}

```

Next a function that computes the 2-norm in double precision for a vector of a generic T type.

```

template<class T>
double norm2_as_double(const hdnum::Vector<T>& v)
{
    double sum = 0.0;

    for (std::size_t i = 0; i < v.size(); ++i) {
        const double vi = scalar_cast<double>(v[i]);
        sum += vi * vi;
    }

    return std::sqrt(sum);
}

```

Then we define a function that encapsulates the steps to solve an equation $Ax = b$ after LU decomposition is done. As input it expects the result of the LU decomposition, the permutation vectors p and q , and the vector b :

```

template<class T>
hdnum::Vector<T>
solve_with_lu_fullpivot(
    const hdnum::DenseMatrix<T>& LU,
    const hdnum::Vector<std::size_t>& p,
    const hdnum::Vector<std::size_t>& q,
    const hdnum::Vector<T>& b
)
{
    hdnum::Vector<T> y(b); // copy b
    hdnum::Vector<T> x(b.size()); // holds solution
}

```

```

    hdnum::permute_forward(p, y);
    hdnum::solveL(LU, y, y); // Solve L*y = P*b, in-place in y.
    hdnum::solveR(LU, x, y); // Solve U*x = y
    hdnum::permute_backward(q, x); // undo column permutation
    return x;
}

```

Then a function that performs the residual computation with the necessary type conversions. It expects a matrix A , vectors b and current solution x , all in working precision u , converts them to higher precisions, and returns the residual $r = Ax - b$. in higher residual precision u_r :

```

template<class T_residual, class T_work>
hdnum::Vector<T_residual> compute_residual(
    const hdnum::DenseMatrix<T_work>& A,
    const hdnum::Vector<T_work>& b,
    const hdnum::Vector<T_work>& x
)
{
    const std::size_t n = b.size();

    hdnum::DenseMatrix<T_residual> A_r(A.rowsize(), A.colsize());
    hdnum::Vector<T_residual> b_r(n);
    hdnum::Vector<T_residual> x_r(n);
    hdnum::Vector<T_residual> Ax_r(n);
    hdnum::Vector<T_residual> r_r(n);

    convert(A_r, A);
    convert(b_r, b);
    convert(x_r, x);

    A_r.mv(Ax_r, x_r);

    for (std::size_t i = 0; i < n; i++) {
        r_r[i] = b_r[i] - Ax_r[i];
    }

    return r_r;
}

```

Finally we provide the full algorithm, where the comments provide the necessary explanations:

```

template <class T_factor, class T_work, class T_residual>
MixedIRResult<T_work> mixed_ir(
    const hdnum::DenseMatrix<T_work>& A,
    const hdnum::Vector<T_work>& b,
    const MixedIROptions& options = {}
)
{
    if (A.rowsize() != A.colsize() || A.rowsize() == 0) {
        throw std::invalid_argument("mixed_ir: A must be square and non-empty");
    }

    if (A.rowsize() != b.size()) {
        throw std::invalid_argument("mixed_ir: A must be square and nonempty");
    }

    const std::size_t n = b.size();

    // if tol is default value 0.0, then use the unit roundoff of T_work
    // which is compute in compile time
    // otherwise use the value passed in the options.
    const double tol =
        options.rel_correction_tol > 0.0 ? options.rel_correction_tol
        : default_unit_roundoff<T_work>();

    // 1. convert A and b to factorization precision.
    hdnum::DenseMatrix<T_factor> A_f(n, n);
    hdnum::Vector<T_factor> b_f(n);

    convert(A_f, A);
    convert(b_f, b);

    // 2. Compute LU factorization in low precision
    //
    // lr_fullpivot overwrites A_f with the combined L / U factors
    hdnum::Vector<std::size_t> p(n);
    hdnum::Vector<std::size_t> q(n);

    hdnum::lr_fullpivot(A_f, p, q);

    // 3. Initial low-precision solve
    hdnum::Vector<T_factor> x0_f = solve_with_lu_fullpivot(A_f, p, q, b_f);

```

```

// 4. cast x0 to working precision
MixedIRResult<T_work> result;
result.x = hdnum::Vector<T_work>(n);
convert(result.x, x0_f);

// 5. refinement loop
for (std::size_t k = 0; k < options.max_iterations; k++) {
    // Compute residual  $r = b - A * x$  in residual precision.
    hdnum::Vector<T_residual> r_r = compute_residual<T_residual>(A, b, result.x);

    // cast residual to working precision
    hdnum::Vector<T_work> r_w(n);
    convert(r_w, r_r);

    // The triangular solve uses T_factor L and U factors,
    // so the right-hand side must also be in T_factor
    hdnum::Vector<T_factor> r_f(n);
    convert(r_f, r_w);

    // Solve correction equation  $A d = r$  using existing LU factors:
    hdnum::Vector<T_factor> d_f = solve_with_lu_fullpivot(A_f, p, q, r_f);

    // Cast correction back to working precision
    hdnum::Vector<T_work> d_w(n);
    convert(d_w, d_f);

    const double norm_d = norm2_as_double(d_w);
    const double norm_x = norm2_as_double(result.x);
    const double denominator = (norm_x > 0.0) ? norm_x : 1.0;

    const double rel_corr = norm_d / denominator;

    result.rel_corrections.push_back(rel_corr);
    result.final_rel_correction = rel_corr;

    // update in working precision
    for (std::size_t i = 0; i < n; i++) {
        result.x[i] += d_w[i];
    }

    result.iterations = k + 1;
}

```

```

        if (norm_d <= tol * norm_x) {
            result.converged = true;
            break;
        }

    }

    return result;
}

```

4.3 Unit Roundoff u

For a floating point format with t significand bits the unit round off is defined as

$$\begin{aligned}
 u &= \frac{1}{2}\beta^{1-t} \\
 &= 2^{-t} \quad (\beta = 2)
 \end{aligned}$$

hdnum's cfloat wrapper's are:

```

using FP16 = hdnum::CPFloat<11,5>;
using bfloat16 = hdnum::CPFloat<8,8>;
using FP8 = hdnum::CPFloat<4,4>;

```

Where the first template parameters are the significand (or the mantisse) bits.

and the FP wrappers:

```

using FP128 = FP<64>;
using FP256 = FP<192>;
using FP512 = FP<448>;
using FP1024 = FP<960>;

```

where the template parameters are the mantisse bits.

From here we can construct the following table summarizing the unit roundoff u for the various precisions:

HDNUM type	Actual HDNUM alias	Significand precision (t)	Unit roundoff $u = 2^{-t}$
FP8	CPFloat<4,4>	4	$2^{-4} \approx 6.25 \cdot 10^{-2}$
bfloat16	CPFloat<8,8>	8	$2^{-8} \approx 3.91 \cdot 10^{-3}$
FP16	CPFloat<11,5>	11	$2^{-11} \approx 4.88 \cdot 10^{-4}$
FP32	float	24	$2^{-24} \approx 5.96 \cdot 10^{-8}$
FP64	double	53	$2^{-53} \approx 1.11 \cdot 10^{-16}$
FP128	FP<64>	64	$2^{-64} \approx 5.42 \cdot 10^{-20}$
FP256	FP<192>	192	$2^{-192} \approx 1.59 \cdot 10^{-58}$
FP512	FP<448>	448	$2^{-448} \approx 1.38 \cdot 10^{-135}$
FP1024	FP<960>	960	$2^{-960} \approx 1.03 \cdot 10^{-289}$

4.3.1 unit roundoff traits

Originally we had the function

```
template<class T>
double default_unit_roundoff()
{
    static_assert(std::numeric_limits<T>::is_specialized,
        "default_unit_roundoff: no numeric_limits specialization for this type;"
        "please set_rel_correction_tol_explicitly");

    return 0.5 * static_cast<double>(std::numeric_limits<T>::epsilon());
}
```

This works only for data types `float` and `double`. We improve this using type traits, s.t. unit roundoffs are returned for other floating point formats at compile time:

```
#pragma once

#include <limits>
#include <type_traits>

#include "hdnum.hh"

namespace mpir {

constexpr double pow2_neg(int p)
{
    double x = 1.0;
```

```

    for (int i = 0; i < p; ++i) {
        x *= 0.5;
    }
    return x;
}

namespace detail {

template<class>
struct unit_roundoff_always_false : std::false_type {};

} // namespace detail

template<class T>
struct unit_roundoff_traits {
    static_assert(detail::unit_roundoff_always_false<T>::value,
        "unit_roundoff_traits: unsupported scalar type");
};

template<>
struct unit_roundoff_traits<hdnum::FP32> {
    static constexpr double value =
        0.5 * static_cast<double>(std::numeric_limits<hdnum::FP32>::epsilon());
};

template<>
struct unit_roundoff_traits<hdnum::FP64> {
    static constexpr double value =
        0.5 * static_cast<double>(std::numeric_limits<hdnum::FP64>::epsilon());
};

#ifdef HDNUM_HAS_CPFLOAT
template<int m, int e>
struct unit_roundoff_traits<hdnum::CPFfloat<m,e>> {
    static constexpr double value = pow2_neg(m);
};
#endif

#ifdef HDNUM_HAS_GMP
template<int m>
struct unit_roundoff_traits<hdnum::FP<m>> {
    static constexpr double value = pow2_neg(m);
};

```

```
};  
#endif  
  
template<class T>  
constexpr double default_unit_roundoff()  
{  
    return unit_roundoff_traits<T>::value;  
}  
  
} // namespace mpir
```

Now mixed ir algorithm will work for all fp formats without having to specify tol manually in the options struct.

5 Implementing the Experiments & Robustness Improvements

We want to plot the **forward error** and the **backward error** as function of the refinement step i .

forward error:

$$\text{forward error}_i = \frac{\|x_i - x_{\text{true}}\|}{\|x_{\text{true}}\|}$$

Backward error:

$$\eta_i = \frac{\|b - Ax_i\|}{\|A\| \|x_i\| + \|b\|}$$

in `mixed_ir.hpp` the result struct currently only stores `rel_correction` vector, that records

$$\frac{\|d_i\|}{\|x_i\|}$$

We have to modify `mixed_ir` to store the iterates x_i , so that we can compute forward and backward errors. We add the following member variable to the `MixedIRResult` struct:

```
std::vector<hdnum::Vector<T_work>> iterates;
```

We append the solutions to the `iterates` vector before the loop and after each loop iteration with the line:

```
if (options.store_iterates) {  
    result.iterates.push_back(result.x);  
}
```

5.1 Temporary Matrix Generator

We start with a simple controlled family, a rotated SPD matrix:

$$A = Q\Lambda Q^T$$

where Λ has eigenvalues $1/\sqrt{\kappa} \dots \sqrt{\kappa}$. Then

$$\kappa_2(A) = \kappa$$

This family of matrices give a known true (exact) solution:

$$x_{\text{true}} = (1, \dots, 1)^T$$

So it can be used temporarily to test the pipeline.

The implementation is in

`code/include/test_matrices.hpp`

5.2 Matrix Generator Supervisor Version

The temporary generator constructs

$$A = Q_{\text{block}}\Lambda Q_{\text{block}}^T$$

where Q_{block} consists only of independent 2×2 rotations. Consequently A is block diagonal:

$$A = \begin{pmatrix} A_1 & & & 0 \\ & A_2 & & \\ & & \ddots & \\ 0 & & & A_m \end{pmatrix}, \quad A_i \in \mathbb{R}^{2 \times 2}.$$

Even though the entire block-diagonal matrix had condition number κ , the individual 2×2 blocks is only approximately:

$$\kappa^{1/(n-1)}$$

Supervisors `randspd` constructs

$$A = Q\Lambda Q^T$$

where Q is a dense random orthogonal matrix. Consequently, A is generally dense, and every variable is coupled to every other variable, producing a genuinely ill-conditioned system.

5.2.1 Nonsymmetric Generator

Iterative refinement with LU is not restricted to SPD matrices, so `randsvd` is also important. The wrapper

```
template<class T>
LinearSystem<T>
make_random_svd_problem(std::size_t n,
                        double kappa,
                        unsigned int seed_u = 42,
                        unsigned int seed_v = 137)
{
    hdnum::DenseMatrix<T> A(n, n);
    hdnum::Vector<T> x_true(n);
    hdnum::Vector<T> b(n);

    hdnum::randsvd(
        A,
        scalar_cast<T>(kappa),
        seed_u,
        seed_v
    );

    for (std::size_t i = 0; i < n; ++i) {
        x_true[i] = T(1);
    }

    A.mv(b, x_true);

    return LinearSystem<T>{
        A,
        b,
        x_true,
        kappa
    };
}
```

produces

$$A = U\Sigma V^T$$

with singular values $1, \dots, 1/\kappa$. It is nonsymmetric and a more general test of the iterative LU algorithm.

5.3 Configurable test-problem Construction

The test-matrix infrastructure contained in ‘test_matrices.hpp’ supports three different ways of constructing the right-hand side b and solution of

$$Ax = b$$

The matrix generator and the right-hand-side strategy are separated:

1. A matrix generator constructs A
2. A shared `complete_problem` function constructs b .
3. The reference solution is computed in a separate, higher precision.

5.3.1 Three Right-hand-side Modes

There are three available strategies represented by:

```
enum class RightHandSideMode {  
    ones_solution,  
    random_sign_solution,  
    random_normal_rhs  
};
```

- `ones_solution`:

$$x = (1, \dots, 1)^T$$
$$b = \text{fl}(Ax)$$

- `random_sign_solution`:

Construct a reproducible random vector with

$$x_i \in \{-1, 1\}$$

and form

$$b = \text{fl}(Ax)$$

- `random_normal_rhs`:

Here we generate the right-hand-side **directly** (randomly):

$$b_i \sim \mathcal{N}(0, 1)$$

In all these cases the reference (true) solution is obtained by solving

$$Ax_{\text{ref}} = b$$

in a high precision (called measure or reference precision).

5.3.2 Test-problem Options

The choices and random seeds are collected in:

```
struct TestProblemOptions {
    RightHandSideMode rhs_mode =
        RightHandSideMode::ones_solution;

    unsigned int matrix_seed_u = 42;
    unsigned int matrix_seed_v = 137;
    unsigned int vector_seed = 2718;

    double rotation_theta = 0.3;
};
```

- `rhs_mode`: selects how b is constructed
- `matrix_seed_u`: seed for the first random orthogonal matrix (used by `randspd`)
- `matrix_seed_v`: seed for the second random orthogonal matrix (used by `randsvd`)
- `vector_seed`: seed for the random-sign solution or random-normal right-hand side.
- `rotation_theta`: rotation angle used by the structured rotated-SPD generator.

The default mode is `ones_solution`.

5.3.3 Linear-system Representation

The system stores the problem and reference solution (potentially in different precisions):

```
template<class T_data, class T_reference = T_data>
struct LinearSystem {
    hdnum::DenseMatrix<T_data> A;
    hdnum::Vector<T_data> b;
    hdnum::Vector<T_reference> x_true;
    double kappa;
};
```

- `T_data`: the precision in which A and b are stored, normally `T_work`.
- `T_reference`: precision used for the reference solution, normally `T_measure`.

5.3.4 Shared `complete_problem` function

We have three matrix generators,

- `make_rotated_spd_problem`
- `make_random_spd_problem`
- `make_random_svd_problem`

each created only A and delegates the remaining work to `create_problem`, the signature:

```
template<class T_data, class T_reference>
LinearSystem<T_data, T_reference>
complete_problem(
    hdnum::DenseMatrix<T_data> A,
    double kappa,
    const TestProblemOptions& options);
```

The logical structure is:

```
if (options.rhs_mode ==
    RightHandSideMode::ones_solution) {

    // Construct  $x = (1, \dots, 1)$ .
    // Compute  $b = A*x$  in T_data.
}
else if (options.rhs_mode ==
    RightHandSideMode::random_sign_solution) {
```

```

    // Construct reproducible x_i in {-1,1}.
    // Compute b = A*x in T_data.
}
else {
    // Construct reproducible b_i ~ N(0,1)
    // directly in T_data.
}

```

After b is constructed the reference solution is computed:

```

auto x_reference =
    high_precision_solve<T_reference>(A, b);

```

i The reference solution is recomputed

Even in the first two modes, even if we explicitly construct x , we recompute

$$x_{\text{true}} = \text{solve}_{T_{\text{ref}}}(A, b)$$

because the matrix-vector product

$$b = \text{fl}(Ax_{\text{constructed}})$$

is founded. Therefore, the exact solution of the stored pair (A, b) can differ slightly from $x_{\text{constructed}}$.

5.3.5 High-precision reference solve

The helper

```

template<class T_reference, class T_data>
hdnum::Vector<T_reference>
high_precision_solve(
    const hdnum::DenseMatrix<T_data>& A,
    const hdnum::Vector<T_data>& b);

```

contained in `reference_solve.hpp` computes a reference solution using `hdnums` LU-solver functions in `T_reference` precision.

For example, `T_reference = hdnum::FP256` provides a sufficiently accurate reference solution.

5.3.6 Matrix-generator interfaces

As mentioned there are three generators, all with compatible interfaces:

```
template<class T_data, class T_reference = T_data>
LinearSystem<T_data, T_reference>
make_rotated_spd_problem(
    std::size_t n,
    double kappa,
    const TestProblemOptions& options = {});
```

```
template<class T_data, class T_reference = T_data>
LinearSystem<T_data, T_reference>
make_random_spd_problem(
    std::size_t n,
    double kappa,
    const TestProblemOptions& options = {});
```

```
template<class T_data, class T_reference = T_data>
LinearSystem<T_data, T_reference>
make_random_svd_problem(
    std::size_t n,
    double kappa,
    const TestProblemOptions& options = {});
```

Each generator

1. constructs its particular matrix A
2. calls `complete_problem`
3. returns `LinearSystem<T_data, T_reference>` struct.

Which enables plugging in different matrix families in an experiment.

5.3.7 Usage Examples

Define the precisions:

```
using T_work = hdnum::FP64;
using T_measure = hdnum::FP256;
```

- all-ones x :

```

mpir::TestProblemOptions problem_options;

problem_options.rhs_mode =
    mpir::RightHandSideMode::ones_solution;

auto problem =
    mpir::make_random_spd_problem<T_work, T_measure>(
        n,
        kappa,
        problem_options
    );

```

- random-sign ones x :

```

mpir::TestProblemOptions problem_options;

problem_options.rhs_mode =
    mpir::RightHandSideMode::random_sign_solution;

problem_options.vector_seed = 2718;

auto problem =
    mpir::make_random_spd_problem<T_work, T_measure>(
        n,
        kappa,
        problem_options
    );

```

- random-normal b :

```

mpir::TestProblemOptions problem_options;

problem_options.rhs_mode =
    mpir::RightHandSideMode::random_normal_rhs;

problem_options.vector_seed = 1;

auto problem =
    mpir::make_random_svd_problem<T_work, T_measure>(
        n,
        kappa,
        problem_options
    );

```

only the generator call needs to change, the resulting problems are used identically:

```
auto result =
    mpir::mixed_ir<T_factor, T_work, T_residual>(
        problem.A,
        problem.b,
        ir_options
    );
```

5.4 Error Metrics Helper

We implement functions to compute

- $\|\cdot\|_\infty$ for matrices and vectors,
- relative forward error with the infinity norm
- relative backward error with the infinity norm

contained in the file

`code/include/error_metrics.hpp`

5.5 First Convergence Test

We test our pipeline with a first test that uses the precision tripple:

- u_f : FP16
- u : FP64
- u_r : FP128

For matrices of size 100×100 with κ values: 1, 10, 100, 1000, 10000.

The test writes for each matrix the convergence history to csv file. Specifically it writes the values:

- kappa,
- iteration,
- relative forward error
- relative backward error
- relative correction

in a single lines for each iteration step, for each matrix. This gives us the convergence histories.

It is implemented in the file

`code/experiments/exp_convergence.cc`

Following was the result of the experiment:

```
kappa,iteration,forward_error_inf,backward_error_inf,rel_correction
1,0,0,0,0
1,1,0,0,0
10,0,0.000976562,0.000441793,0
10,1,9.53674e-07,4.31651e-07,0.00047591
10,2,5.96046e-08,5.1214e-09,3.1641e-07
10,3,5.96046e-08,5.1214e-09,1.68587e-08
10,4,5.96046e-08,5.1214e-09,1.68587e-08
10,5,5.96046e-08,5.1214e-09,1.68587e-08
10,6,5.96046e-08,5.1214e-09,1.68587e-08
10,7,5.96046e-08,5.1214e-09,1.68587e-08
10,8,5.96046e-08,5.1214e-09,1.68587e-08
10,9,5.96046e-08,5.1214e-09,1.68587e-08
10,10,5.96046e-08,5.1214e-09,1.68587e-08
10,11,5.96046e-08,5.1214e-09,1.68587e-08
10,12,5.96046e-08,5.1214e-09,1.68587e-08
10,13,5.96046e-08,5.1214e-09,1.68587e-08
10,14,5.96046e-08,5.1214e-09,1.68587e-08
10,15,5.96046e-08,5.1214e-09,1.68587e-08
10,16,5.96046e-08,5.1214e-09,1.68587e-08
10,17,5.96046e-08,5.1214e-09,1.68587e-08
10,18,5.96046e-08,5.1214e-09,1.68587e-08
10,19,5.96046e-08,5.1214e-09,1.68587e-08
10,20,5.96046e-08,5.1214e-09,1.68587e-08
100,0,0.000976562,0.000481752,0
100,1,9.53674e-07,4.70694e-07,0.000444832
100,2,1.78814e-07,1.42242e-09,2.52178e-07
100,3,1.78814e-07,1.42242e-09,0
1000,0,0.000976562,0.000460731,0
1000,1,9.53674e-07,3.07722e-07,0.000591975
1000,2,8.9407e-07,6.88472e-10,4.02364e-07
1000,3,8.9407e-07,5.65293e-10,1.3328e-08
1000,4,8.9407e-07,5.65293e-10,1.19209e-08
```

```

1000,5,8.9407e-07,5.65293e-10,1.19209e-08
1000,6,8.9407e-07,5.65293e-10,1.19209e-08
1000,7,8.9407e-07,5.65293e-10,1.19209e-08
1000,8,8.9407e-07,5.65293e-10,1.19209e-08
1000,9,8.9407e-07,5.65293e-10,1.19209e-08
1000,10,8.9407e-07,5.65293e-10,1.19209e-08
1000,11,8.9407e-07,5.65293e-10,1.19209e-08
1000,12,8.9407e-07,5.65293e-10,1.19209e-08
1000,13,8.9407e-07,5.65293e-10,1.19209e-08
1000,14,8.9407e-07,5.65293e-10,1.19209e-08
1000,15,8.9407e-07,5.65293e-10,1.19209e-08
1000,16,8.9407e-07,5.65293e-10,1.19209e-08
1000,17,8.9407e-07,5.65293e-10,1.19209e-08
1000,18,8.9407e-07,5.65293e-10,1.19209e-08
1000,19,8.9407e-07,5.65293e-10,1.19209e-08
1000,20,8.9407e-07,5.65293e-10,1.19209e-08
10000,0,0.000976562,0.000375237,0
10000,1,3.33786e-06,4.73515e-08,0.000488379
10000,2,9.53674e-07,3.8223e-10,5.28939e-07
10000,3,9.53674e-07,1.29324e-10,1.03238e-08
10000,4,9.53674e-07,1.29324e-10,0

```

5.6 Robustness Improvements to `mixed_ir`

We encountered two failures at $\kappa(A) = 10^9$. Both caused a floating-point exception, but for different reasons.

5.6.1 1. FP16–FP64–FP128: overflow during input conversion

Problem

We converted A and b from FP64 to the FP16 factorization precision. FP16 has a maximum finite value of approximately

65504.

Some entries exceeded this limit. The converted data therefore contained non-finite values. LU factorization and the initial solve propagated these values into x_0 .

The program crashed when `compute_residual` attempted to convert the non-finite iterate to the GMP-backed FP128 type. This type cannot represent NaN or infinity.

Detection

The GDB backtrace located the crash in the GMP conversion performed by `compute_residual`. We traced the invalid values backward:

$$A_f, b_f \longrightarrow LU_f \longrightarrow x_{0,f} \longrightarrow x_0 \longrightarrow \text{GMP conversion.}$$

Inspection showed that the first non-finite values appeared during conversion to FP16.

The bfloat16 experiment supported this conclusion. Bfloat16 has a wider exponent range and did not overflow at the same point.

Solution

We chose to detect the overflow and terminate safely. We did not implement scaling or equilibration.

The algorithm now terminates with `factorization_input_non_finite` if A_f or b_f contains a non-finite value.

Scaling could prevent this specific overflow, but it would not change $\kappa(A)$. It would therefore not make FP16 iterative refinement reliable at $\kappa(A) = 10^9$.

5.6.2 2. FP32–FP64–FP128: divergence of iterative refinement

Problem

FP32 could represent the matrix entries, so the initial conversion succeeded. The refinement process nevertheless diverged because

$$\kappa(A)u_f \approx 10^9 \cdot 6 \times 10^{-8} \approx 60 > 1.$$

The absolute correction norms grew by a factor of approximately 36 per iteration.

The existing stopping quantity,

$$\frac{\|d_k\|_2}{\|x_k\|_2},$$

did not detect this behavior. Both d_k and x_k grew at similar rates, so the ratio remained approximately constant at 37.29.

Without additional checks, the growth eventually produced non-finite values and another unsafe conversion to FP128.

Detection

We examined the ratio of consecutive correction norms:

$$\frac{\|d_k\|_2}{\|d_{k-1}\|_2} \approx 36.$$

This showed persistent, rapid growth. We therefore decided to monitor absolute correction norms instead of relying only on relative corrections.

Solution

We added a divergence detector based on

$$\|d_k\|_2 > \gamma \|d_{k-1}\|_2.$$

The default parameters are

$$\gamma = 10, \quad \text{required growth steps} = 3.$$

The algorithm reports divergence after three consecutive excessive-growth comparisons. It rejects the correction that triggers termination and preserves the last accepted iterate.

5.6.3 Source-Code Changes

Termination statuses

We introduced statuses that distinguish the main outcomes:

```
enum class MixedIRStatus {
    converged,
    max_iterations,
    factorization_input_non_finite,
    non_finite,
    diverged
};
```

These represent:

- successful convergence;
- reaching the iteration limit;
- non-finite factorization input;
- a non-finite value produced later;
- detected divergence.

Finite-value checks

We added `all_finite` helpers for vectors and matrices.

IEEE-like types and CPFloat values are checked with `std::isfinite`. GMP-backed `hdnum::FP<m>` values require separate handling because they cannot represent NaN or infinity.

The main rule is:

Check IEEE-like values before converting them to a GMP-backed type.

We kept all checks inside `mixed_ir`. We did not modify `HDNUM`, `lowprec_cpfloat.hh`, `compute_residual`, or the triangular solvers.

Guard placement

We added finite-value checks at the following points:

Location	Purpose
After converting A, b to A_f, b_f	Detect factorization-input overflow
After the initial solve	Detect an invalid initial solution
After converting $x_{0,f}$ to working precision	Validate the initial iterate
Before <code>compute_residual</code>	Protect the GMP conversion
After computing r_r	Support generic residual types
After converting r_r to r_f	Detect narrowing overflow

Location	Purpose
After solving for d_f	Detect an invalid correction
After converting d_f to d_w	Validate the working-precision correction
After converting the relative correction to double	Detect norm or conversion overflow
After forming x_{k+1}	Detect overflow during the update

We now form the new iterate as a candidate:

```
hdnum::Vector<T_work> x_next(result.x);

for (std::size_t i = 0; i < n; ++i) {
    x_next[i] += d_w[i];
}

if (!all_finite(x_next)) {
    result.status = MixedIRStatus::non_finite;
    return result;
}

result.x = x_next;
```

This preserves the last valid iterate if the update fails. We increment `result.iterations` only after accepting the update.

Divergence options

We added three options:

```
bool detect_divergence = true;

T_work divergence_growth_factor = T_work(10);

std::size_t divergence_growth_steps = 3;
```

They control:

- whether divergence detection is enabled;
- the minimum excessive-growth factor;
- the required number of consecutive growth steps.

Valid user-supplied values should satisfy:

```
divergence_growth_factor > T_work(1)
divergence_growth_steps > 0
```

Correction-norm history

We added diagnostic storage to `MixedIRResult`:

```
std::vector<double> correction_norms;
```

It stores each finite value of $\|d_k\|_2$.

The history also includes a correction that triggers divergence but is not applied. We store the norms even when divergence detection is disabled.

Detector state and comparison

Before the refinement loop, we introduced:

```
T_work previous_correction_norm = T_work(0);
bool have_previous_correction_norm = false;
std::size_t consecutive_growth_steps = 0;
```

After computing and validating d_w , we calculate:

```
const T_work correction_norm_w = hdnum::norm(d_w);
```

We then test for excessive growth:

```
const bool excessive_growth =
    previous_correction_norm > T_work(0)
    &&
    correction_norm_w
        > options.divergence_growth_factor
        * previous_correction_norm;
```

Excessive growth increments the counter. Any other result resets it. Once the counter reaches the configured limit, the algorithm returns

```
result.status = MixedIRStatus::diverged;
```

before applying the correction.

5.6.4 Result

With divergence detection enabled, the FP32–FP64–FP128 experiment terminates early with `diverged`.

With divergence detection disabled, the finite-value guards still prevent a crash. In our test, the algorithm completed 19 updates before detecting a non-finite value:

```
kappa = 1e+09  
converged = 0  
iterations = 19  
final_rel_correction = 37.2908
```

The two safeguards have separate roles:

- The divergence detector stops persistent growth early.
- The finite-value checks prevent unsafe operations and crashes.

5.6.5 FP16 residual underflow and stagnation problem

We tested three-precision iterative refinement with

$$T_{\text{factor}} = \text{FP16}, \quad T_{\text{work}} = \text{FP64}, \quad T_{\text{residual}} = \text{FP128}.$$

The LU factorization and correction solve used FP16. Iterates were stored and updated in FP64. Residuals were computed in FP128.

For $n = 100$, the convergence sweep produced unexpected results:

- For $\kappa = 10$, the method did not converge after 20 iterations. The forward error stagnated near 5.96×10^{-8} , and the relative correction stagnated near 1.69×10^{-8} .
- For $\kappa = 100$, the method appeared to converge after only three iterations because the computed correction became exactly zero.
- Similar nonmonotone behavior occurred for larger condition numbers.

This was inconsistent with the usual refinement condition. Since

$$u_{\text{FP16}} = 2^{-11} \approx 4.88 \times 10^{-4},$$

we have, for $\kappa = 10$,

$$\kappa u_{\text{FP16}} \approx 4.88 \times 10^{-3} \ll 1.$$

The $\kappa = 10$ problem should therefore have been favorable for iterative refinement.

The repeated values

$$5.96046 \times 10^{-8} \quad \text{and} \quad 1.19209 \times 10^{-7}$$

were an important clue. These are respectively the smallest positive FP16 subnormal number and twice that number.

Empirical diagnosis

Comparison with bfloat16

We replaced FP16 with bfloat16. Bfloat16 has fewer significand bits than FP16 but a much larger exponent range.

The relevant results were:

κ	Converged	Iterations	Final relative correction
1	Yes	2	5.07389×10^{-17}
10	Yes	14	4.71855×10^{-17}
100	No	20	6.68688×10^{-15}
1000	No	20	2.24847
10000	No	20	12.5079

In particular, bfloat16 converged for $\kappa = 10$, while FP16 did not. Because bfloat16 has lower precision but greater range, this supported the hypothesis that the FP16 failure was caused by its restricted exponent range, not by insufficient significand precision alone.

Conceptual hypothesis

At iteration k , the residual is computed accurately in FP128:

$$r_k = b - Ax_k.$$

The original implementation then converted it directly to FP16:

$$r_k^{(128)} \longrightarrow r_k^{(16)}.$$

For FP16,

$$x_{\min, \text{normal}} = 2^{-14} \approx 6.1035 \times 10^{-5},$$

and

$$x_{\min, \text{subnormal}} = 2^{-24} \approx 5.9605 \times 10^{-8}.$$

Under round-to-nearest, values below approximately

$$2^{-25} \approx 2.9802 \times 10^{-8}$$

round to zero.

We therefore hypothesized that, as refinement reduced the residual:

1. Some FP128 residual components became zero when converted to FP16.
2. Surviving components were rounded to a small number of FP16 subnormal values.
3. The FP16 correction solve received an increasingly inaccurate right-hand side.
4. The correction eventually stagnated or became exactly zero.

An exactly zero correction could also cause false convergence because the relative-correction stopping criterion would evaluate to zero.

Diagnostic implementation

We added per-iteration residual-conversion diagnostics to `mixed_ir`:

- $\|r_k\|_\infty$, computed in residual precision;
- the smallest nonzero component of r_k ;
- the number of nonzero FP128 residual components;
- the number of nonzero components that became exactly zero during FP128-to-FP16 conversion.

The important distinction was:

- `nonzero_components` described the original FP128 residual;
- `zeroed_by_conversion` described information lost in the FP16 copy.

Thus,

```
nonzero_components = 100
zeroed_by_conversion = 20
```

meant that all 100 components were nonzero in FP128, but only 80 remained nonzero after conversion to FP16.

Diagnostic experiment

We tested one fixed random symmetric positive-definite system with

$$n = 100, qquad \kappa(A) = 10,$$

using FP16–FP64–FP128 refinement.

The initial diagnostic results were:

Iteration	$\ r_k\ _\infty$	Smallest nonzero component	Zeroed by FP16 conversion
0	1.3474×10^{-2}	3.3279×10^{-5}	0
1	5.4754×10^{-5}	3.4203×10^{-8}	0
2	4.8336×10^{-7}	3.0833×10^{-9}	9
3	4.1084×10^{-7}	2.0158×10^{-9}	20

The first two refinement steps reduced the residual substantially:

$$\frac{\|r_0\|_\infty}{\|r_1\|_\infty} \approx 246, \quad \frac{\|r_1\|_\infty}{\|r_2\|_\infty} \approx 113.$$

At iteration 2, residual components first became zero during conversion. At essentially the same point, residual reduction stopped.

For $k \geq 2$,

$$2.97 \times 10^{-7} \leq \|r_k\|_\infty \leq 5.00 \times 10^{-7},$$

with mean approximately

$$4.17 \times 10^{-7}.$$

Between 9 and 20 components were lost per iteration, with an average of approximately 14.5.

Even before components became zero, conversion was already inaccurate. At iteration 1, the smallest component was

$$3.4203 \times 10^{-8}.$$

It was above the rounding-to-zero threshold but below the smallest FP16 subnormal, so it was rounded to approximately

$$5.9605 \times 10^{-8}.$$

Therefore, `zeroed_by_conversion = 0` did not imply an accurate conversion.

After iteration 2, the entire residual was only several FP16 subnormal increments away from zero. The residual was consequently represented by a very small set of possible FP16 values. This produced severe quantization even for components that remained nonzero.

The experiment confirmed the hypothesis:

The FP128 residual remained fully nonzero, but conversion to unscaled FP16 discarded or severely quantized its components. This information loss began at the same point as residual stagnation.

Implemented solution

Conceptual solution

We introduced infinity-norm residual scaling:

$$\theta_k = \|r_k\|_\infty, \quad \hat{r}_k = \frac{r_k}{\theta_k}.$$

Instead of solving directly with the small residual, we solve

$$A\hat{d}_k = \hat{r}_k$$

in factor precision and then recover the actual correction:

$$d_k = \theta_k \hat{d}_k.$$

In exact arithmetic, this is equivalent to

$$Ad_k = r_k.$$

The normalization ensures

$$\|\hat{r}_k\|_\infty = 1.$$

The residual conversion therefore uses the available FP16 range effectively rather than placing the entire vector near zero.

Scaling does not increase FP16 precision. It removes the irrelevant absolute magnitude of the residual before conversion.

Implementation in `mixed_ir`

We added an optional setting:

```
bool scale_residual = false;
```

The default preserves the original algorithm.

When scaling is enabled, the implementation:

1. Computes $\|r_k\|_\infty$ in `T_residual`.

2. Preserves the original residual `r_r` for diagnostics.

3. Creates a separate correction right-hand side:

```
hdnum::Vector<T_residual> correction_rhs_r(r_r);
```

4. Normalizes it when the residual is nonzero:

```
correction_rhs_r[i] =  
    correction_rhs_r[i] / residual_scale_r;
```

5. Converts the normalized vector, rather than the original residual, to factor precision:

```
convert(r_f, correction_rhs_r);
```

6. Performs the existing FP16 LU solve without changing the factors or solve routine.

7. Converts the normalized correction to working precision.

8. Rescales it in FP64:

```
d_w[i] = residual_scale_w * d_w[i];
```

9. Performs the original update:

$$x_{k+1} = x_k + d_k.$$

The scale is applied in working precision, not factor precision. Thus, the small value θ_k never needs to be represented in FP16.

The zero-residual case skips normalization, preventing division by zero. We also retained the existing finite-value checks after rescaling.

The diagnostic semantics were updated accordingly:

- residual magnitudes still describe the original r_k ;
- **zeroed_by_conversion** describes conversion of the actual correction right-hand side:
 - r_k in unscaled mode;
 - $\hat{r}_k$ in scaled mode.

Validation experiment

Experimental design

We generated the problem once and ran:

```
run("unscaled", false);  
run("scaled", true);
```

Both runs used exactly the same:

- matrix;
- right-hand side;
- random seed;
- dimension $n = 100$;
- condition number $\kappa(A) = 10$;
- precision triple FP16–FP64–FP128;
- stopping criterion and iteration limit.

The identical initial residual,

$$\|r_0\|_\infty = 1.34744462924774981 \times 10^{-2},$$

confirmed that scaling did not alter the matrix factorization or initial solution.

The unscaled results also reproduced the previous baseline. Therefore, adding the optional scaling path did not change the original behavior when scaling was disabled.

Unscaled result

The unscaled method:

- first lost components at iteration 2;
- lost between 9 and 20 components per later iteration;
- stagnated near 4.17×10^{-7} ;
- failed to converge after 20 updates;
- ended with

$$\text{final relative correction} = 1.1685 \times 10^{-7}.$$

Scaled result

The scaled run produced:

Iteration	$\ r_k\ _\infty$	Zeroed by conversion	Reduction factor
0	1.3474×10^{-2}	0	–
1	5.3874×10^{-5}	0	250.1
2	2.8349×10^{-7}	0	190.0
3	2.1471×10^{-9}	0	132.0
4	1.1636×10^{-11}	0	184.5
5	7.5515×10^{-14}	0	154.1
6	4.3479×10^{-16}	0	173.7

No normalized residual component became zero during FP16 conversion.

The residual decreased by

$$\frac{\|r_0\|_\infty}{\|r_6\|_\infty} \approx 3.10 \times 10^{13},$$

or approximately 13.5 decimal orders.

The method converged after seven updates with

$$\text{final relative correction} = 9.7687 \times 10^{-17}.$$

This was below FP64 unit roundoff:

$$u_{\text{FP64}} = 2^{-53} \approx 1.1102 \times 10^{-16}.$$

Interpretation

The observed reduction factors, between approximately 132 and 250, correspond to contraction factors between roughly

$$4.0 \times 10^{-3} \quad \text{and} \quad 7.6 \times 10^{-3}.$$

This agrees well with the theoretical scale

$$\kappa(A)u_{\text{FP16}} \approx 4.88 \times 10^{-3}.$$

Once the residual-range problem was removed, the algorithm behaved as expected from mixed-precision iterative-refinement theory.

Conclusion

The investigation established the following sequence:

1. FP16 refinement failed unexpectedly for a well-conditioned problem.
2. Bfloat16 succeeded despite having lower significand precision, pointing to an exponent-range problem.
3. Direct diagnostics showed that nonzero FP128 residual components were zeroed or severely quantized during FP16 conversion.
4. Conversion loss began at the same point as residual stagnation.
5. Infinity-norm scaling removed the irrelevant absolute residual magnitude before conversion.
6. A controlled scaled-versus-unscaled experiment eliminated all observed conversion-to-zero events.
7. The scaled method restored regular convergence and reached FP64 accuracy in seven iterations.

This is strong validation for the tested system. Broader validation should repeat the comparison over multiple dimensions, condition numbers, and random seeds. Residual scaling addresses residual underflow only; it does not address separate FP16 overflow caused by matrix entries exceeding the FP16 range.

5.7 Plotting Convergence Experiments

The script

```
code/scripts/plot_convergence.py
```

generates convergence-history and summary plots from one mixed-precision iterative-refinement CSV file.

It is designed to be reusable for different:

- matrix generators;
- right-hand-side or solution-generation modes;

- precision combinations.

The input CSV is selected using a command-line argument, so a separate plotting script is not needed for each experiment.

5.7.1 Expected CSV format

The CSV file must contain the columns:

```
kappa
iteration
forward_error_inf
backward_error_inf
rel_correction
converged
total_iterations
final_rel_correction
```

Each row describes one iterate for one condition number.

The usual relationship is:

```
iteration 0 → initial solution x0
iteration 1 → solution x1 after the first correction
iteration 2 → solution x2 after the second correction
...
```

5.7.2 Generated plots

For every input CSV, the script generates six plots.

Convergence histories

1. Relative forward error against refinement iteration.
2. Normwise backward error against refinement iteration.
3. Relative correction norm against refinement iteration.

Each condition number is displayed as a separate curve.

Summary plots

4. Final forward and backward errors against condition number.
5. Best forward and backward errors attained at any iteration.
6. Total refinement iterations against condition number.

The best-attained-error plot is useful when an iteration initially improves and later deteriorates. For example, a difficult ($\kappa=10000$) problem may attain its best forward error around iteration 7 but have a worse result at iteration 20.

The script also prints a numerical summary containing:

- condition number;
- convergence flag;
- total iterations;
- final forward error;
- final backward error;
- final relative correction;
- best forward and backward errors;
- iterations at which the best errors occurred.

5.7.3 Logarithmic plots and zero values

The error histories use logarithmic vertical axes. Since zero cannot be displayed on a logarithmic scale, the script replaces nonpositive plotting values with NaN.

This affects only plotting. The original CSV data is not modified.

Consequently, a curve or point with an exactly zero error may disappear from a logarithmic plot.

5.7.4 Command-line interface

The general invocation is:

```
python code/scripts/plot_convergence.py CSV_FILE \  
    --label "Description of experiment"
```

If the terminal is already inside `code/`, use:

```
python scripts/plot_convergence.py CSV_FILE \  
    --label "Description of experiment"
```

The script first checks whether `CSV_FILE` is a direct path. If it is not found there, it looks under:

```
code/results/raw/
```

The plots are written to:

```
code/results/plots/
```

5.7.5 Default behavior

The CSV argument is optional. If no filename is provided, the script uses:

```
convergence_fp16_fp64_fp128.csv
```

Therefore:

```
python code/scripts/plot_convergence.py
```

is equivalent to:

```
python code/scripts/plot_convergence.py \  
    convergence_fp16_fp64_fp128.csv
```

5.7.6 Output tags

The output tag is normally derived automatically from the CSV filename.

For example, from:

```
convergence_random_spd_fp16_fp64_fp128.csv
```

the script removes the initial `convergence_` and obtains:

```
random_spd_fp16_fp64_fp128
```

It then generates filenames such as:

```
convergence_forward_error_random_spd_fp16_fp64_fp128.png
convergence_backward_error_random_spd_fp16_fp64_fp128.png
convergence_rel_correction_random_spd_fp16_fp64_fp128.png
summary_final_errors_random_spd_fp16_fp64_fp128.png
summary_best_errors_random_spd_fp16_fp64_fp128.png
summary_iterations_random_spd_fp16_fp64_fp128.png
```

A custom output tag can be supplied with:

```
--tag custom_name
```

For example:

```
python code/scripts/plot_convergence.py \
    convergence_random_spd_fp16_fp64_fp128.csv \
    --tag random_spd_test
```

5.7.7 Descriptive plot labels

The optional `--label` argument is added to plot titles:

```
--label "Random dense SPD"
```

This produces titles such as:

```
Random dense SPD: relative forward error, infinity norm vs. refinement iteration
```

The label does not affect the output filenames.

5.7.8 Matrix-family examples

Block-rotated SPD

```
python code/scripts/plot_convergence.py \
    convergence_fp16_fp64_fp128.csv \
    --label "Block-rotated SPD"
```

Random dense SPD

```
python code/scripts/plot_convergence.py \  
    convergence_random_spd_fp16_fp64_fp128.csv \  
    --label "Random dense SPD"
```

Random SVD

```
python code/scripts/plot_convergence.py \  
    convergence_random_svd_fp16_fp64_fp128.csv \  
    --label "Random SVD"
```

5.7.9 Encoding the solution or RHS mode

The test-problem infrastructure supports three modes:

```
RightHandSideMode::ones_solution  
RightHandSideMode::random_sign_solution  
RightHandSideMode::random_normal_rhs
```

These correspond to:

- construct $(x=(1,\dots,1)^T)$ and form $(b=Ax)$;
- construct random $(x_i \in [-1,1])$ and form $(b=Ax)$;
- generate $(b_i \sim N(0,1))$ and compute a high-precision reference solution.

The third mode uses a normal distribution, not a uniform distribution.

It is sensible to encode this choice in the CSV filename. A consistent naming scheme is:

```
convergence_<matrix-family>_<rhs-mode>_<factor>_<work>_<residual>.csv
```

For example:

```
convergence_random_spd_x_ones_fp16_fp64_fp128.csv  
convergence_random_spd_x_random_signs_fp16_fp64_fp128.csv  
convergence_random_spd_b_random_normal_fp16_fp64_fp128.csv
```

The plotting script automatically retains the RHS mode as part of the output tag. No code change is needed.

5.7.10 Examples for the three modes

All-ones solution

```
python code/scripts/plot_convergence.py \  
    convergence_random_spd_x_ones_fp16_fp64_fp128.csv \  
    --label "Random SPD, ones solution"
```

This produces files such as:

```
convergence_forward_error_random_spd_x_ones_fp16_fp64_fp128.png
```

Random-sign solution

```
python code/scripts/plot_convergence.py \  
    convergence_random_spd_x_random_signs_fp16_fp64_fp128.csv \  
    --label "Random SPD, random-sign solution"
```

This produces files such as:

```
convergence_forward_error_random_spd_x_random_signs_fp16_fp64_fp128.png
```

Random-normal right-hand side

```
python code/scripts/plot_convergence.py \  
    convergence_random_spd_b_random_normal_fp16_fp64_fp128.csv \  
    --label "Random SPD, normally distributed right-hand side"
```

This produces files such as:

```
convergence_forward_error_random_spd_b_random_normal_fp16_fp64_fp128.png
```

5.7.11 What the script detects automatically

The script automatically derives an output tag from the filename. It does not semantically interpret components such as:

```
random_spd
x_ones
x_random_signs
b_random_normal
fp16_fp64_fp128
```

Therefore:

- output filenames are generated correctly automatically;
- the same plots are generated for every mode;
- a human-readable title should currently be supplied through `--label`.

Automatic semantic interpretation could be added later, but it is not necessary for the current workflow.

5.7.12 Recording metadata inside the CSV

Encoding the experiment configuration in the filename is useful, but the CSV should eventually also record metadata such as:

```
matrix_family
rhs_mode
matrix_seed_u
matrix_seed_v
vector_seed
```

For example:

```
matrix_family,rhs_mode,matrix_seed_u,matrix_seed_v,vector_seed,kappa,...
random_spd,x_random_signs,42,137,2718,10,...
```

This makes the experiment self-describing even if the file is renamed.

5.7.13 Later comparison plots

`plot_convergence.py` plots one CSV dataset at a time. It should remain focused on that task.

A later script, for example

```
code/scripts/plot_compare_generators.py
```

or

```
code/scripts/plot_compare_rhs_modes.py
```

can place multiple experiments on the same figures.

Possible comparisons include:

- block-rotated SPD versus random SPD versus random SVD;
- all-ones versus random-sign solutions;
- prescribed (x) versus random-normal (b);
- different precision triples;
- different random seeds.

Thus, the intended organization is:

```
plot_convergence.py
    → detailed plots for one experiment

plot_compare_generators.py
    → compare matrix families

plot_compare_rhs_modes.py
    → compare solution/RHS-generation strategies
```

6 Mixed-Precision Iterative Refinement: Experiment Plan

With the corrected third task and the decision to keep the primary condition sweeps deterministic, the complete plan becomes the following.

The best place to begin remains the experiment-output infrastructure. Once the full suite is running, incomplete metadata, inconsistent filenames, or unclear seed handling would force us to regenerate results.

6.1 Overall experiment plan

The experiments can be divided into the four objectives from the project brief and a fifth group covering our FP16 robustness extension.

Group	Purpose	Main variables	Main outputs
A. Convergence histories	Examine refinement iteration by iteration	precision triple, selected $\kappa(A)$	forward error, backward error, relative correction
B. Condition-number sweeps	Locate the convergence boundary	$\kappa(A)$, precision triple	final/best errors, iterations, status
C. Residual-precision comparison	Validate the last two rows of Table 7.1	single–double–double vs. single–double–quad	forward and backward error vs. $\kappa(A)$
D. Direct-solve comparison	Measure the benefit of refinement	mixed IR vs. direct solve in working precision	error vs. $\kappa(A)$
E. FP16 robustness extension	Demonstrate range checks and residual scaling	scaling on/off, $\kappa(A)$	histories, conversion diagnostics, termination status

For the primary experiments, the default random seeds should remain fixed. This isolates the effect of the condition number and keeps the plots straightforward.

6.2 Group A: convergence histories

These experiments show how the algorithm behaves over individual refinement iterations.

6.2.1 Representative precision triples

A suitable selection is:

- FP64–FP64–FP64: same-precision baseline;
- FP32–FP64–FP64: classical mixed precision;
- FP32–FP64–FP128: higher residual precision;
- FP16–FP64–FP64: half-precision factorization;
- FP16–FP64–FP128: principal FP16 configuration;
- bfloat16–FP64–FP128: exponent-range comparison;
- FP8–FP64–FP128: optional stress test.

Not every precision triple needs to appear in the main report. A representative subset can be selected after inspecting the results.

6.2.2 Condition numbers

For each factorization precision, choose condition numbers:

- clearly inside the predicted convergence region;
- near $\kappa(A)u_f \approx 1$;
- outside the predicted region.

6.2.3 Plots

For every selected configuration, plot against iteration:

1. relative forward error;
2. normwise backward error;
3. relative correction,

$$\frac{\|d_k\|_2}{\|x_k\|_2}.$$

The seeds remain fixed within and across comparisons. This ensures that different precision triples solve exactly the same problems.

6.3 Group B: general condition-number sweeps

These experiments study how each precision configuration behaves as conditioning deteriorates.

Use a logarithmic condition-number grid, supplemented with additional points near the expected convergence boundary $1/u_f$.

For example:

- FP16: concentrate around 10^2 – 10^4 ;
- bfloat16: concentrate around 10^1 – 10^3 ;
- FP32: concentrate around 10^6 – 10^9 .

6.3.1 Seed policy

Only $\kappa(A)$ should vary. Use the default fixed seeds for:

- the matrix generator;
- the right-hand side;
- any generated orthogonal transformations.

Provided the generator is reset deterministically for each value of $\kappa(A)$, this keeps the matrix orientation and right-hand side controlled while changing the prescribed spectrum.

Multiple seeds are not required. They can be added later only if the primary sweep produces unexpectedly irregular results.

A concise report formulation is:

The random seeds were fixed throughout the sweep. Thus, the matrix structure and right-hand side remained controlled while the prescribed condition number was varied. This isolates the effect of conditioning and permits direct comparisons between precision configurations.

6.3.2 Recorded quantities

For each run, record:

- requested condition number;
- measured condition number;
- termination status;
- final forward and backward errors;
- best attained forward and backward errors;

- iterations at which the best errors occurred;
- number of completed refinement updates;
- final relative correction.

6.3.3 Plots

Create:

1. final forward error vs. $\kappa(A)$;
2. final backward error vs. $\kappa(A)$;
3. best attained error vs. $\kappa(A)$;
4. iteration count vs. $\kappa(A)$;
5. termination status vs. $\kappa(A)$.

Show the theoretical condition

$$\kappa(A)u_f = 1$$

as a vertical reference line.

Because the seeds are fixed, each configuration produces a single curve. No medians, ranges, or quantile bands are needed.

6.4 Group C: single–double–double versus single–double–quad

This replaces the earlier broad residual-precision study. It directly implements the experiment requested by the supervisor and validates the final two rows of Table 7.1 in the Carson–Higham paper.

6.4.1 Precision configurations

Compare:

1. single–double–double:

$$u_f = u_{\text{single}}, \quad u = u_r = u_{\text{double}};$$

2. single–double–quad:

$$u_f = u_{\text{single}}, \quad u = u_{\text{double}}, \quad u_r = u_{\text{quad}} \approx u^2.$$

Only the residual precision changes. Both variants must use:

- the same matrix;
- the same right-hand side;
- the same fixed seeds;
- the same stopping criterion;
- the same maximum iteration count;
- the same remaining algorithm options.

6.4.2 Theoretical expectation

The general limiting forward-error bound is

$$\frac{\|x - \hat{x}\|_\infty}{\|x\|_\infty} \lesssim 4p u_r \operatorname{cond}(A, x) + u.$$

For single-double-double, $u_r = u$, giving

$$4p u \operatorname{cond}(A, x) + u.$$

The conditioning-dependent term generally dominates, so the expected forward error is

$$O(u \operatorname{cond}(A, x)).$$

For single-double-quad, $u_r \approx u^2$, giving

$$4p u^2 \operatorname{cond}(A, x) + u.$$

Within the convergence region of the single-precision factorization, the first term is sufficiently small that the working-precision term dominates. Therefore, the expected forward error is

$$O(u).$$

Both variants should attain a backward error of order

$$O(u).$$

Thus, the quad residual should improve the limiting forward error substantially, while both variants should produce approximately double-precision backward errors.

The remembered equation

$$4pu^2 \operatorname{cond}(A, x) + u \lesssim 4pu$$

is correct, but equation (8.5) appears in the later GMRES-IR analysis. For ordinary LU-IR, the general bound above provides the more direct explanation.

6.4.3 Condition-number sweep

A suitable initial range is approximately

$$10^0 \leq \kappa(A) \leq 10^9,$$

with additional points between 10^6 and 10^8 , near the single-precision convergence boundary.

For IEEE single precision,

$$u_f = 2^{-24} \approx 5.96 \times 10^{-8}, \quad u_f^{-1} \approx 1.68 \times 10^7.$$

This is only an approximate boundary. The actual transition can occur earlier because it also depends on the stability of the computed factorization.

6.4.4 Required plots

The supervisor requested two log–log plots, both with condition number on the horizontal axis:

1. forward error for both variants in one plot;
2. backward error for both variants in one plot.

Expected behaviour:

Plot	Expected result
Forward error	single–double–double grows approximately as $u \operatorname{cond}(A, x)$; single–double–quad remains near u
Backward error	both variants remain near u while refinement converges

Useful reference lines are:

- u , representing the double-precision floor;
- $u\kappa(A)$, illustrating the expected single–double–double trend;
- $1/u_f$, marking the approximate convergence boundary.

6.4.5 Measurement details

Table 7.1 uses $\kappa_\infty(A)$, whereas the forward-error bound contains $\text{cond}(A, x)$. They are related but not identical.

Each run should therefore record:

- requested/generated condition number;
- measured $\kappa_2(A)$;
- preferably measured $\kappa_\infty(A)$;
- $\text{cond}(A, x)$, if practical;
- forward and backward errors;
- termination status;
- iteration count.

The main horizontal axis should preferably use measured $\kappa_\infty(A)$, matching Table 7.1. Error measurements should use sufficiently high precision, ideally FP256.

This experiment can be summarized as:

A condition-number sweep compares single–double–double and single–double–quad iterative refinement. Both variants are expected to attain double-precision backward error. The quad-residual variant should also attain double-precision forward error, whereas the double-residual variant should attain forward error of order $u \text{cond}(A, x)$.

6.5 Group D: comparison with a direct solve

For every chosen working precision u , compare:

- a single LU solve performed entirely in u ;
- mixed-precision iterative refinement whose iterates are stored in u .

The principal question is whether mixed IR attains working-precision accuracy while performing the factorization in a cheaper, lower precision.

6.5.1 Plots

Create:

1. relative forward error vs. $\kappa(A)$;
2. backward error vs. $\kappa(A)$;
3. optionally, iteration count for mixed IR.

Both methods must solve identical problems with fixed seeds.

6.6 Group E: FP16 robustness extension

This should form a separate report section because it goes beyond the original project requirements.

Two distinct FP16 range problems should be presented separately:

- factorization-input overflow;
- residual-conversion underflow.

Residual scaling mitigates the second problem. It does not repair overflow in the original factorization input.

6.6.1 E1. Controlled scaling comparison

Use the existing controlled problem:

- random SPD matrix;
- $n = 100$;
- $\kappa(A) = 10$;
- random-normal right-hand side;
- FP16–FP64–FP128;
- identical default seeds;
- residual scaling off and on.

Only `scale_residual` should change.

6.6.2 Principal plots

Plot:

1. forward error vs. iteration;
2. backward error vs. iteration;
3. relative correction vs. iteration;
4. residual infinity norm vs. iteration.

6.6.3 Diagnostic plots

Also plot:

5. number of residual components zeroed during conversion to FP16;
6. smallest nonzero residual component vs. iteration, with the FP16 subnormal threshold shown.

This experiment should demonstrate the observed mechanism:

- without scaling, increasingly many residual components are lost during conversion and the algorithm stagnates;
- with scaling, the residual is normalized before conversion;
- conversion-to-zero is avoided or greatly reduced;
- the algorithm reaches approximately FP64 accuracy.

6.6.4 E2. Broader scaling validation

A simple broader validation can also keep the default seeds fixed and vary only $\kappa(A)$. Compare scaling on and off over several condition numbers for which the FP16 factorization input remains finite.

Record:

- termination status;
- final and best forward errors;
- iteration count;
- maximum number of zeroed residual components;
- total number of zeroing events;
- residual-reduction factor.

Create:

1. final forward error vs. $\kappa(A)$;
2. iterations vs. $\kappa(A)$;

3. maximum zeroed-component count vs. $\kappa(A)$;
4. termination status vs. $\kappa(A)$.

Multiple seeds are optional here as well. They would strengthen a claim of general robustness, but they are not necessary for the primary report. They should only be added if time permits or if the fixed-seed results appear atypical.

6.6.5 E3. FP16 overflow handling

Input overflow should be demonstrated separately.

Record:

$\kappa(A)$	Maximum $ A_{ij} $	FP16 input finite	Termination status
-------------	--------------------	-------------------	--------------------

The expected robust behaviour is termination with a status such as `factorization_input_non_finite`, rather than a crash or propagation of invalid values.

A table or categorical status plot is more suitable than a convergence-history plot.

6.7 Controlling the experiment space

Testing every combination of matrix family, right-hand side, precision triple, condition number, seed, dimension, and scaling mode would produce an unnecessarily large experiment suite.

A focused selection is preferable:

- primary matrix family: random SPD;
- primary right-hand side: random-normal;
- primary dimension: $n = 100$;
- fixed default seeds;
- rotated SPD: supplementary controlled case;
- random SVD: supplementary nonsymmetric case;
- alternative right-hand sides: optional sensitivity study;
- multiple seeds: optional robustness check only.

This keeps the report centred on the requested effects.

6.8 Common metadata design

The experimental utilities should be based on one shared experiment description.

Conceptually:

```
struct PrecisionNames {
    std::string factor;
    std::string work;
    std::string residual;
    std::string measure;
};

struct ExperimentDescription {
    ExperimentKind kind;
    MatrixFamily matrix_family;
    std::size_t dimension;
    PrecisionNames precisions;
};
```

The metadata writer should additionally receive:

- TestProblemOptions;
- MixedIROptions<T_work>;
- the condition number or condition-number grid;
- the variant name, such as `scaled` or `unscaled`.

6.8.1 Experiment identity

Record:

- experiment kind;
- matrix family;
- dimension n ;
- factorization precision;
- working precision;
- residual precision;
- measurement precision;
- algorithm variant.

6.8.2 Test-problem options

Record:

- right-hand-side mode;
- `matrix_seed_u`;
- `matrix_seed_v`;
- `vector_seed`;
- `rotation_theta`;
- prescribed condition number;
- measured condition numbers.

Recording the seeds is important even though they remain fixed. It makes the deterministic problem construction reproducible.

6.8.3 Algorithm options

Record:

- `max_iterations`;
- effective relative-correction tolerance;
- `detect_divergence`;
- `divergence_growth_factor`;
- `divergence_growth_steps`;
- `store_iterates`;
- `scale_residual`;
- `record_residual_diagnostics`.

The effective tolerance should be written, not merely a sentinel value such as a nonpositive number meaning “use the default.”

6.8.4 Results

Record:

- termination status;
- convergence flag, if retained;
- number of completed updates;
- final relative correction;
- final forward and backward errors;
- best forward and backward errors;
- iterations of the best errors.

6.8.5 CSV representation

I recommend repeating invariant metadata as ordinary columns in every CSV row. This is redundant but useful:

- every file remains self-contained;
- `pandas.read_csv()` works without a custom parser;
- files can be concatenated directly;
- the experiment remains identifiable if the file is renamed.

A JSON sidecar containing the Git revision, compiler information, timestamp, and build configuration could be added later, but it is not required for the first refactoring.

6.9 Automatic filenames

The filename should encode the experiment identity, but not every option.

A suitable pattern is:

```
<experiment>__<matrix>__<rhs>__uf-<factor>__u-<work>__ur-<residual>__<variant>.csv
```

Examples:

```
convergence__random-spd__random-normal__uf-fp32__u-fp64__ur-fp128.csv
```

```
residual-precision__random-spd__random-normal__uf-fp32__u-fp64__ur-comparison.csv
```

```
residual-scaling__random-spd__random-normal__uf-fp16__u-fp64__ur-fp128__comparison.csv
```

The filename generator and metadata writer should consume the same `ExperimentDescription`. This prevents the filename and CSV contents from disagreeing.

Precision names can initially be supplied explicitly. They can later be generated through traits such as `precision_name<T>()`.

6.10 Output-directory organization

A moderate hierarchy is sufficient:

```
results/  
  raw/  
    convergence/  
    condition_sweeps/  
    residual_precision/  
    direct_comparison/  
    robustness/  
      residual_scaling/  
      range_failures/  
  plots/  
    convergence/  
    condition_sweeps/  
    residual_precision/  
    direct_comparison/  
    robustness/  
      residual_scaling/  
      range_failures/
```

Plotting scripts should mirror the raw-data directory structure. This avoids accumulating many ambiguously named plots in one flat directory.

6.11 Recommended implementation order

1. Define the experiment kinds, metadata schema, seed policy, filename convention, and directory layout.
2. Implement `experiment_io.hpp` with:
 - enum-to-string conversions;
 - metadata-header writing;
 - metadata-value writing;
 - filename generation;
 - output-path construction.
3. Refactor the residual-scaling comparison as the pilot experiment.
4. Ensure residual diagnostics distinguish:
 - the original residual;
 - the normalized residual actually passed to the conversion;
 - values lost during conversion.
5. Add a dedicated scaled-versus-unscaled plotting script.
6. Refactor the existing convergence-history experiment.

7. Implement the general condition-number sweep with fixed default seeds.
8. Implement the targeted single-double-double versus single-double-quad sweep.
9. Implement the direct-solve comparison.
10. Run the broader fixed-seed FP16 scaling validation.
11. Add multiple-seed tests only if they become scientifically useful and time permits.

The immediate next step therefore remains the shared experiment-description and output utilities. The residual-scaling comparison is the best first script to refactor because it applies two algorithm configurations to exactly the same deterministic test problem.

6.11.1 Implementation Plan

Start with one small vertical slice: build the shared output utility and use it only in the residual-scaling comparison. Once that works, extend it to the other experiments.

6.11.2 1. Create the branch

If you have not already done so:

```
git switch -c feature/experiment-infrastructure
```

6.11.3 2. Freeze the common schema first

Before implementing functions, decide which information every experimental row should contain.

I suggest separating it into three groups.

Experiment identity

```
experiment
matrix_family
dimension
factor_precision
work_precision
residual_precision
measure_precision
variant
```

Problem and algorithm configuration

```
rhs_mode
matrix_seed_u
matrix_seed_v
vector_seed
rotation_theta
requested_kappa
max_iterations
effective_rel_correction_tol
detect_divergence
divergence_growth_factor
divergence_growth_steps
store_iterates
scale_residual
record_residual_diagnostics
```

Run result

```
status
converged
total_iterations
final_rel_correction
```

Experiment-specific columns—such as `iteration`, `forward_error_inf`, or `zeroed_by_conversion`—come after these common columns.

The important distinction is:

- metadata describes how the run was configured;
- result fields describe how it terminated;
- measurement fields depend on the experiment.

6.11.4 3. Create `experiment_io.hpp`

The first version only needs the following components:

```

enum class ExperimentKind {
    convergence_history,
    condition_sweep,
    residual_precision,
    direct_solve_comparison,
    residual_scaling,
    range_failure
};

enum class MatrixFamily {
    random_spd,
    rotated_spd,
    random_svd
};

struct PrecisionNames {
    std::string factor;
    std::string work;
    std::string residual;
    std::string measure;
};

struct ExperimentDescription {
    ExperimentKind kind;
    MatrixFamily matrix_family;
    std::size_t dimension;
    PrecisionNames precisions;
};

```

Then implement these utility groups:

```

std::string_view to_string(ExperimentKind kind);
std::string_view to_string(MatrixFamily family);
std::string_view to_string(RightHandSideMode mode);
std::string_view to_string(MixedIRStatus status);

```

```

std::filesystem::path make_output_directory(
    const std::filesystem::path& raw_results_root,
    ExperimentKind kind
);

```

```
std::string make_experiment_filename(
    const ExperimentDescription& experiment,
    const TestProblemOptions& problem_options,
    std::string_view tag = {}
);
```

```
void write_common_csv_header(std::ostream& out);
```

```
template<class T_work>
void write_common_csv_fields(
    std::ostream& out,
    const ExperimentDescription& experiment,
    const TestProblemOptions& problem_options,
    const MixedIROptions<T_work>& algorithm_options,
    double requested_kappa,
    std::string_view variant,
    const MixedIRResult<T_work>& result
);
```

The last function should write the values in exactly the same order as `write_common_csv_header()`.

6.11.5 4. Use the residual-scaling experiment as the pilot

Refactor `exp_residual_scaling_fp16_fp64_fp128(1).cc` first.

Define its identity once:

```
const mpir::ExperimentDescription experiment{
    mpir::ExperimentKind::residual_scaling,
    mpir::MatrixFamily::random_spd,
    n,
    {"fp16", "fp64", "fp128", "fp256"}
};
```

Generate its path rather than hard-coding it:

```
const auto output_dir =
    mpir::make_output_directory(
        MPIR_RESULTS_RAW_DIR,
        experiment.kind
    );
```

```
const auto output_file =
    output_dir /
    mpir::make_experiment_filename(
        experiment,
        problem_options,
        "comparison"
    );
```

The expected result would be something like:

```
results/raw/robustness/residual_scaling/
residual-scaling__random-spd__random-normal__uf-fp16__u-fp64__ur-fp128__comparison.csv
```

Inside the two runs, set:

```
run("unscaled", false);
run("scaled", true);
```

The metadata should then show that every option is identical except:

```
variant
scale_residual
```

This makes the controlled nature of the comparison explicit.

6.11.6 5. Verify the pilot carefully

Check four things:

1. The automatically generated directory exists.
2. The generated filename matches the actual precision types.
3. Every CSV row contains the complete configuration.
4. The numerical results are unchanged from the old experiment.

That final check is important: this refactoring should change only output organization, not algorithm behaviour.

The current convergence experiment demonstrates exactly why automatic naming is useful: [exp_convergence_random_spd\(3\).cc](#) declares `T_factor = FP32`, but its hard-coded filename says `fp16`.

6.11.7 6. Keep the first milestone narrow

For the first milestone, do only:

- `experiment_io.hpp`;
- common enum-to-string functions;
- common metadata columns;
- filename generation;
- output-directory generation;
- refactoring of the residual-scaling experiment.

Leave these for the following commits:

- refining `ResidualDiagnostic`;
- plotting scaled versus unscaled histories;
- refactoring all other experiments;
- implementing the condition-number sweeps.

A suitable first commit sequence would be:

```
Add common experiment metadata and naming utilities
Refactor residual-scaling experiment to use shared output utilities
```

So the immediate next action is to create `experiment_io.hpp` and define the two enums, `PrecisionNames`, and `ExperimentDescription`. That gives us the vocabulary on which all subsequent output functions will depend.

6.11.8 Condition Grids

. The two groups need related but distinct condition-number grids:

- Group A needs a small representative set: clearly below, near, and above the theoretical boundary.
- Group B needs a denser logarithmic sweep to observe where convergence actually breaks down.

The reference boundary should be computed from the factorization precision:

$$\kappa_* = \frac{1}{u_f},$$

not from the working precision u . The condition $\kappa(A)u_f < 1$ is the approximate convergence condition for standard iterative refinement.

I would introduce two utilities rather than manually listing condition numbers.

```
template<class T_factor>
std::vector<double> representative_kappas();

template<class T_factor>
std::vector<double> kappa_sweep(
    double min_kappa = 1.0,
    double upper_boundary_factor = 10.0,
    std::size_t points_per_decade = 10
);
```

The first could generate condition numbers relative to κ_* , for example:

```
1,
0.01 *,
0.1 *,
0.5 *,
*,
2 *,
10 *
```

after sorting and removing duplicates. This provides:

- a well-conditioned baseline;
- points clearly inside the theoretical region;
- points close to the boundary;
- several points outside it.

The dense sweep could logarithmically cover

$$1 \leq \kappa(A) \leq 10\kappa_*,$$

with perhaps 8–12 points per decade. We may later add extra points around κ_* , such as

$$0.5\kappa_*, \quad 0.8\kappa_*, \quad \kappa_*, \quad 1.25\kappa_*, \quad 2\kappa_*,$$

if the ordinary logarithmic grid is not sufficiently dense there.

One important qualification is that this boundary is approximate:

1. Our generator may prescribe $\kappa_2(A)$, whereas the theoretical result may use $\kappa_\infty(A)$.

2. The actual convergence boundary also depends on the computed LU factors and their stability.
3. A generated point satisfying $\kappa u_f < 1$ is therefore not guaranteed to converge, and a point slightly above it is not guaranteed to fail.

That is precisely why the larger sweep is useful: the utility centers the experiment around the theoretical prediction, while the experiment identifies the empirical transition.

I would place these functions in a separate shared header such as:

```
code/include/condition_grids.hpp
```

They are numerical experiment-design utilities, whereas `experiment_io.hpp` should remain concerned with filenames, directories, metadata, and CSV output. Keeping those responsibilities separate will make the structure clearer.

We should use one source file for Group A, not one file per precision triple.

Each precision triple is a compile-time type combination, so the Group A driver will define one templated runner and instantiate it for each selected triple. Each instantiation writes a separate CSV containing all representative condition numbers and all iterations.

6.11.9 Further Details

```
code/
  CMakeLists.txt
  include/
    mixed_ir.hpp
    experiment_io.hpp
    condition_grids.hpp
    ...
  experiments/
    CMakeLists.txt
    convergence/
      CMakeLists.txt
      exp_convergence_histories.cc
    condition_sweeps/
      CMakeLists.txt
      exp_condition_sweeps.cc
    residual_precision/
    direct_comparison/
    robustness/
```

```

    CMakeLists.txt
    exp_residual_scaling.cc
    exp_range_failure.cc
scripts/
    convergence/
        plot_convergence_histories.py
results/
    raw/
        convergence/
    plots/
        convergence/

```

Descriptive directory names are preferable to `group_a`, `group_b`, and so on. The letters belong to our current report plan; the experiment purposes are more stable.

The existing residual-scaling driver can eventually be moved into `experiments/robustness/`.

6.11.10 One template, several precision triples

The central function will have approximately this interface:

```

template<
    class T_factor,
    class T_work,
    class T_residual,
    class T_measure
>
void run_convergence_histories(
    const mpir::PrecisionNames& precision_names,
    const std::vector<double>& kappas
);

```

It will perform the complete experiment for one precision triple:

1. Construct its `ExperimentDescription`.
2. Generate its output directory and filename.
3. Write the CSV header.
4. Loop over the supplied condition numbers.
5. Generate the test problem with fixed seeds.
6. Run iterative refinement with `store_iterates = true`.
7. Measure forward and backward error at every stored iterate.
8. Write one CSV row per iterate.

Then `main()` invokes it explicitly:

```
int main()
{
    using hdnum::FP8;
    using hdnum::FP16;
    using hdnum::FP32;
    using hdnum::FP64;
    using hdnum::FP128;
    using hdnum::FP256;
    using hdnum::bfloat16;

    run_convergence_histories<
        FP64, FP64, FP64, FP256
    >(
        {"fp64", "fp64", "fp64", "fp256"},
        fp64_baseline_kappas
    );

    run_convergence_histories<
        FP32, FP64, FP64, FP256
    >(
        {"fp32", "fp64", "fp64", "fp256"},
        mpir::representative_kappas<FP32>()
    );

    run_convergence_histories<
        FP32, FP64, FP128, FP256
    >(
        {"fp32", "fp64", "fp128", "fp256"},
        mpir::representative_kappas<FP32>()
    );

    run_convergence_histories<
        FP16, FP64, FP64, FP256
    >(
        {"fp16", "fp64", "fp64", "fp256"},
        mpir::representative_kappas<FP16>()
    );

    run_convergence_histories<
        FP16, FP64, FP128, FP256
    >(
```

```

    {"fp16", "fp64", "fp128", "fp256"},
    mpir::representative_kappas<FP16>()
);

run_convergence_histories<
    bfloat16, FP64, FP128, FP256
>(
    {"bfloat16", "fp64", "fp128", "fp256"},
    mpir::representative_kappas<bfloat16>()
);

// Optional stress test.
run_convergence_histories<
    FP8, FP64, FP128, FP256
>(
    {"fp8", "fp64", "fp128", "fp256"},
    mpir::representative_kappas<FP8>()
);
}

```

The precise implementation will use the actual HDNUM aliases, but this is the intended structure.

6.11.11 Condition-number selection

For FP8, bfloat16, FP16, and FP32, we can use:

```

mpir::representative_kappas<T_factor>()

```

This produces points clearly below, near, and above

$$\kappa_* = \frac{1}{u_f}.$$

Configurations with the same factorization precision automatically receive the same condition numbers. For example, FP32–FP64–FP64 and FP32–FP64–FP128 will solve identical problems, allowing a direct residual-precision comparison.

The FP64 baseline requires special treatment. Its boundary is approximately

$$\frac{1}{u_{\text{FP64}}} \approx 9 \times 10^{15},$$

which is too close to the numerical resolution limit of an FP64-stored test matrix. We should therefore use a moderate baseline grid, for example:

```
const std::vector<double> fp64_baseline_kappas{
    1.0,
    1.0e4,
    1.0e8,
    1.0e12
};
```

The FP64 run is a same-precision accuracy baseline, not an attempt to locate its full convergence boundary.

6.11.12 Experiment settings

The initial Group A dataset should use:

```
const std::size_t n = 100;

mpir::TestProblemOptions problem_options;
problem_options.rhs_mode =
    mpir::RightHandSideMode::random_normal_rhs;
```

The default seeds remain unchanged, ensuring that a given κ produces the same matrix and right-hand side for every precision triple.

The algorithm options should be fixed:

```
mpir::MixedIROptions<T_work> options;
options.max_iterations = 20;
options.store_iterates = true;
options.detect_divergence = true;
options.record_residual_diagnostics = false;
options.scale_residual = true;
```

I recommend enabling residual scaling in the main Group A experiments because it is now part of our robust implementation. Group E retains the controlled scaled-versus-unscaled comparison that demonstrates why it is needed. The CSV metadata records this choice explicitly.

6.11.13 CSV organization

Each precision triple gets one CSV containing all its condition numbers and iteration histories.

For example:

```
results/raw/convergence/  
  convergence-history__random-spd__n-100__random-normal__uf-fp64__u-fp64__ur-fp64__um-fp256.  
  convergence-history__random-spd__n-100__random-normal__uf-fp32__u-fp64__ur-fp64__um-fp256.  
  convergence-history__random-spd__n-100__random-normal__uf-fp32__u-fp64__ur-fp128__um-fp256.  
  ...
```

Each file will have the common columns followed by:

```
iteration,forward_error_inf,backward_error_inf,rel_correction
```

For iteration zero, `rel_correction` should preferably be empty rather than 0, because no correction precedes the initial solution. Writing zero would create an artificial value that is problematic on a logarithmic plot.

6.11.14 Plot organization

For each precision triple, the plotting script should produce one figure with three panels:

1. relative forward error versus iteration;
2. normwise backward error versus iteration;
3. relative correction versus iteration.

Each condition number appears as a separate curve. We can later select the clearest subset of precision triples and condition numbers for the report without rerunning the experiments.

6.11.15 CMake organization

The parent file becomes:

```
## code/experiments/CMakeLists.txt  
  
add_subdirectory(convergence)  
add_subdirectory(condition_sweeps)  
add_subdirectory(residual_precision)  
add_subdirectory(direct_comparison)  
add_subdirectory(robustness)
```

For Group A:

```
## code/experiments/convergence/CMakeLists.txt

add_executable(
    exp_convergence_histories
    exp_convergence_histories.cc
)

target_link_libraries(
    exp_convergence_histories
    PRIVATE
        mixed_precision_core
        mixed_precision_warnings
)
```

The top-level code/CMakeLists.txt only needs:

```
add_subdirectory(experiments)
```

We can keep `mixed_precision_core` as an INTERFACE library: adding experiment `.cc` files does not require converting the core into a compiled library.

So the next concrete step is to refactor the old `exp_convergence_random_spd.cc` into `exp_convergence_histories.cc` with the templated runner above.

6.11.16 How to run

Implemented and verified the new plotting script:

[plot_convergence_histories.py](#)

Place it under `code/scripts/`, then run from `code/`:

```
python3 scripts/plot_convergence_histories.py
```

It automatically:

- Finds all `convergence-history_*.csv` files under `results/raw/convergence/`.
- Generates one three-panel figure per precision configuration.
- Plots each condition number as a separate curve.
- Uses logarithmic vertical axes.

- Handles the empty iteration-0 relative correction correctly.
- Writes figures to `results/plots/convergence/` using the corresponding CSV filename.
- Validates the CSV schema and warns about unplotable status-only runs.

PNG is the default. Vector PDF output is available with:

```
python3 scripts/plot_convergence_histories.py --format pdf
```

You can also plot a particular file:

```
python3 scripts/plot_convergence_histories.py \
    results/raw/convergence/convergence-history_...csv
```

6.11.17 Legends Codes

- [C] — converged
- [M] — maximum iterations
- [D] — diverged
- [N] — non-finite value
- [F] — non-finite factorization input

7 Experimental Framework

a modular, configurable, and reproducible experimental framework for mixed-precision iterative refinement.

The codebase now has several distinct layers:

- The numerical kernel implements mixed IR independently of any particular experiment. The factorization, working, and residual precisions are independent template parameters.
- Runtime options configure stopping criteria, iteration limits, divergence detection, residual scaling, iterate storage, and diagnostic collection.
- Structured results record the final solution, termination status, correction history, iterates, and residual-conversion diagnostics. Numerical failures therefore become observable experimental outcomes instead of crashes or ambiguous non-convergence.
- The test-problem layer separates matrix generation from right-hand-side construction. Matrix family, condition number, dimension, random seeds, and right-hand-side strategy can be changed independently. A higher-precision reference solution is generated through the same interface.
- Shared utilities provide error metrics, condition-number grids, precision boundaries, filenames, output directories, and CSV serialization.
- Experiment drivers mainly select a problem family, precision configuration, parameter grid, and requested measurements. They do not need to reimplement the numerical or output machinery.
- Plotting scripts consume the resulting uniform, self-describing datasets and can therefore compare different experiments consistently.

The most important architectural property is separation of concerns:

```
[ problem construction  ;;  algorithm execution  ;;  measurement  ;;  data output  ;;  
visualization.]
```

Each stage has a reasonably stable interface. Consequently, introducing a new experiment usually means composing existing components rather than constructing a new pipeline. This is what allowed us to implement convergence histories, condition-number sweeps, residual-precision comparisons, direct-solve comparisons, and residual-scaling diagnostics relatively efficiently.

My evaluation is that this is considerably stronger than a collection of ad hoc experiment programs. It has become small domain-specific research software with four particularly valuable properties:

- Reproducibility: configurations, seeds, precision roles, stopping options, and statuses are recorded with the results.
- Comparability: experiments use common problem generators, error definitions, meta-data, and output conventions.
- Extensibility: new precision combinations, matrix families, algorithm variants, and measurements can be added without changing the entire codebase.
- Diagnostic value: the framework records not only whether the algorithm converged, but also why it stopped and what happened internally.

It is still a research framework rather than a general-purpose production library. Some driver boilerplate remains, precision names are supplied separately from the C++ types, and the CSV schema is maintained manually. A larger project might introduce typed experiment records, automatic precision-name traits, configuration files, and more automated tests. For the scope of this practical project, however, the current level of abstraction is appropriate: it improves reliability and reproducibility without burying the numerical algorithm under excessive infrastructure.

A succinct report-ready description would be:

During the project, the implementation evolved into a modular experimental framework for mixed-precision iterative refinement. The framework separates test-problem construction, algorithm configuration, numerical execution, error measurement, result serialization, and visualization. Matrix families, right-hand-side strategies, precision combinations, condition-number grids, stopping criteria, robustness mechanisms, and recorded diagnostics can be configured independently. Shared metadata and CSV utilities produce uniform, self-describing datasets, enabling experiments to be reproduced and compared consistently. New experiments can therefore be implemented mainly by composing existing components rather than duplicating numerical and data-processing code.

A C++ record type defines the data structurally. A schema-aware CSV writer uses that type to generate the header and serialize each row.

Introducing only the record type would improve organization, but it would not automatically remove manual serialization. We need both parts.

7.1 Where the current design stands

We already have typed objects for the inputs and outputs of an experiment:

- `ExperimentDescription`
- `TestProblemOptions`
- `MixedIROptions<T_work>`

- `MixedIRResult<T_work>`
- `ResidualDiagnostic`

But a CSV row is not currently represented by one object. It exists only implicitly in code such as:

```
write_common_csv_header(out);
out << ",iteration,forward_error_inf,backward_error_inf,"
      "rel_correction\n";
```

and later:

```
write_common_csv_fields(...);
out << ',' << iteration
    << ',' << forward_error
    << ',' << backward_error
    << ',' << rel_correction
    << '\n';
```

This produces an important weakness: the header and the row are maintained separately. If a column is inserted, removed, or reordered in one place but not the other, the program still compiles and may silently produce a malformed dataset.

So we already have typed algorithm results, but we do not yet have typed serialized experiment records.

7.2 What a typed experiment record would be

For example, the Group E error-history output could be represented explicitly:

```
struct RunRecord {
    ExperimentKind experiment;
    MatrixFamily matrix_family;
    std::size_t dimension;

    PrecisionNames precisions;
    RightHandSideMode rhs_mode;

    std::uint64_t matrix_seed_u;
    std::uint64_t matrix_seed_v;
    std::uint64_t vector_seed;
```

```

    double requested_kappa;
    std::string variant;

    std::size_t max_iterations;
    double effective_rel_correction_tol;
    bool scale_residual;

    MixedIRStatus status;
    std::size_t total_iterations;
    std::optional<double> final_rel_correction;
};

struct ErrorHistoryRecord {
    RunRecord run;

    std::size_t iteration;
    double forward_error_inf;
    double backward_error_inf;
    std::optional<double> rel_correction;
};

```

The use of `std::optional` is valuable. For example:

- iteration (0) has no associated correction;
- a failed run may have no solution and therefore no forward error;
- a direct solve has no refinement iteration count or stopping tolerance.

Currently these cases are represented by manually emitting empty commas. With typed records, missingness becomes explicit and checked by the type system.

The driver would construct records:

```

records.push_back({
    .run = make_run_record(
        experiment,
        problem_options,
        algorithm_options,
        requested_kappa,
        variant,
        result
    ),
    .iteration = iteration,
});

```

```

        .forward_error_inf = forward_error,
        .backward_error_inf = backward_error,
        .rel_correction =
            iteration == 0
            ? std::nullopt
            : std::optional{
                result.rel_corrections[iteration - 1]
            }
    });

```

The experiment still decides which quantities to measure, but it no longer decides how CSV fields are ordered or escaped.

7.3 How the automatic CSV schema would work

C++ does not normally expose member names through ordinary language facilities, so a record alone cannot automatically produce meaningful column names. We would define a CSV schema once for each record type:

```

template<>
struct CsvSchema<ErrorHistoryRecord> {
    static constexpr auto columns()
    {
        return std::tuple{
            column(
                "iteration",
                &ErrorHistoryRecord::iteration
            ),
            column(
                "forward_error_inf",
                &ErrorHistoryRecord::forward_error_inf
            ),
            column(
                "backward_error_inf",
                &ErrorHistoryRecord::backward_error_inf
            ),
            column(
                "rel_correction",
                &ErrorHistoryRecord::rel_correction
            )
        }
    }
};

```

```

    };
}
};

```

A generic writer could then use the same list of column descriptors for both operations:

```

write_csv_header<ErrorHistoryRecord>(out);
write_csv_row(out, record);

```

Or, preferably:

```

write_csv(output_file, records);

```

Internally, it would:

1. obtain the column names from `CsvSchema<Record>`;
2. write the header;
3. obtain each value through the corresponding accessor;
4. escape strings consistently;
5. represent `std::nullopt` as an empty field;
6. apply uniform numeric formatting.

Because column names and value accessors come from the same descriptor list, they cannot become reordered independently.

The common run metadata could be flattened automatically before the experiment-specific fields:

```

RunRecord columns
+
ErrorHistoryRecord columns

```

This would eliminate code such as:

```

write_common_csv_header(...);
write_common_csv_fields(...);
write_direct_common_fields(...);

```

and the repeated `out << ',' << ...` expressions in every driver.

The schema would still be declared manually once per record type, but not separately in every experiment driver.

7.4 Recommended record types

I would not create one enormous `ExperimentRecord` containing every possible measurement. That would produce many meaningless empty columns.

Instead, use a small family of records:

```
RunRecord
SummaryRecord
IterationRecord
ResidualDiagnosticRecord
DirectComparisonRecord
```

For example:

- `SummaryRecord`: final errors and total iterations for condition sweeps;
- `IterationRecord`: forward error, backward error, and relative correction;
- `ResidualDiagnosticRecord`: residual norm, minimum component, nonzero count, and conversion-loss count;
- `DirectComparisonRecord`: method name, final errors, and method-specific status.

All would embed the same `RunRecord`. This preserves uniform metadata without forcing every dataset into the same schema.

7.5 What configuration files would provide

At present, experiment configuration is compiled into the drivers:

```
constexpr std::size_t problem_dimension = 100;
constexpr double requested_kappa = 500.0;

algorithm_options.max_iterations = 20;
algorithm_options.scale_residual = true;
```

A configuration file would move these choices out of C++ source code. For example, a TOML configuration could look like:

```

schema_version = 1
experiment = "residual-scaling"
variant = "scaled"

[problem]
matrix_family = "random-spd"
dimension = 100
rhs_mode = "random-normal"
requested_kappa = 500.0
matrix_seed_u = 1234
matrix_seed_v = 5678
vector_seed = 9012

[precisions]
factor = "fp16"
work = "fp64"
residual = "fp128"
measure = "fp256"

[algorithm]
max_iterations = 20
rel_correction_tol = "default"
detect_divergence = true
divergence_growth_factor = 10.0
divergence_growth_steps = 3
scale_residual = true
store_iterates = true
record_residual_diagnostics = true

[output]
record_type = "error-history"
tag = "scaled"

```

A single executable could then run:

```
mpir-run experiments/residual_scaling_scaled.toml
```

The advantages would be:

- experiments can be changed without recompilation;
- the exact configuration can be archived with the results;
- related variants can be generated from nearly identical files;

- batch sweeps can be described declaratively;
- one generic runner can replace several narrowly specialized drivers.

The configuration should be parsed into strongly typed structures:

```
struct ProblemConfig;
struct PrecisionConfig;
struct AlgorithmConfig;
struct OutputConfig;
struct ExperimentConfig;
```

Strings such as "random-spd" should be converted immediately into enums. The program should validate the complete configuration before starting the experiment.

7.5.1 One complication: precisions are compile-time types

The numerical call is templated:

```
mixed_ir<T_factor, T_work, T_residual>(...)
```

A configuration file is read at runtime. Therefore, it cannot create arbitrary C++ template types dynamically.

The runner would need a registry of supported, precompiled combinations:

```
dispatch_precision_configuration(config, [&<class Tf, class Tw,
                                     class Tr, class Tm>() {
    run_experiment<Tf, Tw, Tr, Tm>(config);
});
```

Conceptually, the dispatcher maps:

```
fp16-fp64-fp128-fp256
-> run<FP16, FP64, FP128, FP256>()

fp32-fp64-fp64-fp256
-> run<FP32, FP64, FP64, FP256>()
```

This is perfectly workable, but it means that configuration files select from supported precision combinations rather than inventing arbitrary compiled types.

7.6 Automatic precision-name traits

As you said, this is the simpler problem. Instead of passing

```
{"fp16", "fp64", "fp128", "fp256"}
```

separately from

```
<hdnum::FP16, hdnum::FP64, hdnum::FP128, hdnum::FP256>
```

we could define:

```
template<class T>
struct PrecisionTraits;

template<>
struct PrecisionTraits<hdnum::FP16> {
    static constexpr std::string_view name = "fp16";
};

template<>
struct PrecisionTraits<hdnum::FP64> {
    static constexpr std::string_view name = "fp64";
};
```

Then:

```
const auto precision_names =
    make_precision_names<
        T_factor,
        T_work,
        T_residual,
        T_measure
    >();
```

This removes a duplicated source of information and prevents accidentally running FP16 while labeling the output as FP32.

7.7 Research framework versus general-purpose library

Typed records and configuration files would make the experimental framework more systematic. They would not, by themselves, turn the numerical implementation into a general-purpose library.

The clean long-term architecture would separate two products:

```
mpir-core
    Numerical algorithm and reusable numerical interfaces

mpir-experiments
    Problem generators, configurations, measurements, CSV output and plots
```

The core library should not need to know about CSV files, filenames, condition-number sweeps, or plot requirements.

The transition toward a general-purpose library would involve the following.

7.7.1 1. Establish a stable numerical API

The public interface should clearly separate:

- algorithm inputs;
- precision roles;
- options;
- results;
- termination statuses;
- diagnostic information.

Internal helpers should be moved out of the public interface where possible.

7.7.2 2. Separate numerical execution from observation

Currently, complete iterate histories are stored when measurements are needed. A more general interface could support an iteration observer:

```
mixed_ir(A, b, options, [&](const IterationState& state) {
    recorder.observe(state);
});
```

This would allow callers to:

- compute error histories;
- log progress;
- implement custom stopping diagnostics;
- avoid storing every iterate;
- use the numerical library without experiment-specific machinery.

7.7.3 3. Abstract the factorization and matrix backend

The current implementation is closely tied to HDNUM matrices, vectors, and full-pivoting LU. A broader library could define concepts or adapter interfaces for:

- matrix and vector types;
- scalar conversion;
- LU factorization;
- triangular solution;
- norm evaluation;
- finiteness checks.

HDNUM would then become one supported backend rather than an assumption throughout the implementation.

This is the largest architectural change and should only be undertaken if support for other numerical backends is genuinely desired.

7.7.4 4. Separate public headers and compiled utilities

The templated numerical kernel can remain header-based. Non-template components can move to .cc files:

```
include/mpir/
  mixed_ir.hpp
  options.hpp
  result.hpp
  status.hpp

src/
  csv_writer.cc
  configuration.cc
  experiment_io.cc
```

The build should eventually provide an installable CMake target:

```
target_link_libraries(my_program PRIVATE mpir::mpir)
```

7.7.5 5. Add systematic tests

A library-quality project would need:

- unit tests for conversions, norms, stopping conditions, and status handling;
- regression tests for known FP16 underflow and overflow cases;
- tests for singular and dimensionally invalid inputs;
- tests that CSV headers and records have identical schemas;
- configuration parsing and validation tests;
- numerical tests over several dimensions, seeds, and condition numbers;
- sanitizer and multi-compiler builds.

7.7.6 6. Version external contracts

CSV columns and configuration formats become external interfaces once plotting scripts and saved experiments depend on them. They should have explicit schema versions:

```
schema_version = 1
```

Changing or renaming columns would then be a deliberate format change rather than an accidental breaking change.

7.7.7 7. Provide packaging and documentation

A general-purpose library also needs:

- installable headers and CMake configuration;
- documented supported scalar types and backends;
- documented numerical assumptions;
- concise examples;
- semantic versioning;
- a clear distinction between public and internal APIs.

7.8 A practical future roadmap

For this project, I would order the work as follows:

1. Automatic precision traits.
2. `RunRecord` plus typed summary, history, and diagnostic records.
3. A schema-aware generic CSV writer.
4. Serialization tests and a `schema_version` column.
5. A strongly typed `ExperimentConfig`.
6. A TOML-driven generic experiment runner.
7. Separation into `mpir-core` and `mpir-experiments`.
8. An observer interface for per-iteration measurements.
9. Only then consider matrix/backend abstraction and installable-library packaging.

The first four steps would remove most remaining driver boilerplate without substantially complicating the project. Configuration files would be the next step if you expect many more experiments.

The decisive transition to a general-purpose library begins at steps 7–9: when the mixed-IR implementation becomes a stable numerical component that can be used independently of our specific experiment infrastructure. Typed records and configuration files primarily mature the research framework surrounding that numerical component.

Part II

Report

8 Mixed-Precision Iterative Refinement

9 Introduction

Modern hardware supports several floating-point formats with different costs, accuracy, and numerical ranges. Low precision can increase throughput and reduce memory traffic and energy consumption, but it also introduces larger rounding errors. Mixed-precision algorithms combine multiple formats to obtain some of the efficiency of low precision while retaining higher-precision accuracy (Higham and Mary 2022).

This report studies mixed-precision iterative refinement for nonsingular linear systems using low precision for LU factorization and higher precision for refinement. Numerical experiments assess convergence and accuracy across precision triples and condition numbers. The study focuses on convergence, attainable accuracy, and iteration counts, rather than wall-clock performance. Several low- and high-precision formats are emulated or implemented in software, so measured runtimes would not represent performance on hardware with native support for these formats.

10 Three-precision iterative refinement

10.1 Algorithm

The implemented method is LU-based three-precision iterative refinement (Higham and Mary 2022):

Algorithm 10.1 Three-precision iterative refinement

Require: Nonsingular system $Ax = b$

- 1: Round A and b to precision u_f
 - 2: Compute $PAQ = LU$ in precision u_f
 - 3: Solve for x_0 using the factors in precision u_f
 - 4: Store x_0 in precision u
 - 5: **for** $i = 0, 1, \dots, i_{\max} - 1$ **do**
 - 6: Compute $r_i = b - Ax_i$ in precision u_r
 - 7: Round r_i to precision u_f
 - 8: Solve for d_i using the factors in precision u_f
 - 9: Store d_i in precision u
 - 10: $x_{i+1} \leftarrow x_i + d_i$ in precision u
 - 11: **if** $\|d_i\|_2 / \|x_i\|_2 < u$ **then**
 - 12: **return** x_{i+1}
 - 13: **end if**
 - 14: **end for**
 - 15: **return** $x_{i_{\max}}$
-

Algorithm 10.1 summarizes the core three-precision method. Our implementation extends the core algorithm with optional residual scaling for limited-range formats, configurable robustness checks, optional iterate histories, optional residual diagnostics, and descriptive termination statuses.

10.2 Precision roles and theoretical expectations

The three precisions control different aspects of iterative refinement. The factorization precision u_f determines the quality of the LU factors and primarily limits convergence. The

working precision u is used for the iterates and updates and sets the target accuracy. The residual precision u_r determines how accurately the residual is computed and influences the attainable forward error.

For a stable low-precision factorization, convergence is expected when

$$\kappa(A)u_f \ll 1.$$

Thus, u_f^{-1} defines an approximate condition-number scale, denoted by

$$\kappa_* = u_f^{-1}.$$

This is not a strict boundary: the actual convergence rate also depends on LU growth, factorization quality, componentwise conditioning, and norm-dependent constants. Higham and Mary (2022, sec. 7.1)

Under suitable convergence conditions, refinement produces a normwise backward error of order u . The limiting relative forward error satisfies approximately

$$\frac{\|\hat{x} - x\|_\infty}{\|x\|_\infty} \lesssim u + 4pu_r \operatorname{cond}(A, x),$$

where p is the maximum number of nonzeros in a row of $[A \ b]$ Carson and Higham (2018, Corollary 3.3). If $u_r = u$, the condition-dependent term generally limits the attainable forward accuracy. If u_r is sufficiently smaller than u , this term becomes negligible and forward errors of order u can be attained.

11 Experimental setup

All experiments use the same framework for problem construction, iterative refinement, error evaluation, and result recording. Individual experiment groups vary only the condition numbers, precision configurations, and algorithmic options required for their objectives.

11.1 Configuration and common settings

Table 11.1 summarizes the settings shared across the experiments. Unless stated otherwise, all runs use these values. Experiment-specific variations are described in the corresponding results sections.

Table 11.1: Common experimental settings.

Parameter	Common setting
Matrix family	Dense random SPD
Dimension	$n = 100$
Right-hand side	$b_i \sim \mathcal{N}(0, 1)$ in FP64
Random seeds	42 for matrix construction; 2718 for the right-hand side
Working precision	FP64
Reference and measurement precision	FP256
Maximum iterations	20
Divergence detection	Enabled; growth factor 10 over three consecutive steps
Residual scaling	Enabled
Iterate histories	Enabled
Residual diagnostics	Disabled

The condition numbers and precision triples vary between experimental groups. Residual scaling normalizes the residual before conversion to the factorization precision and rescales the computed correction. It is disabled only for the scaled-unscaled comparison and is discussed later under [Low-precision limitations and residual scaling](#).

11.2 Test problems

The experiments use dense random-SPD matrices of the form

$$A = Q\Lambda Q^T,$$

where Q is orthogonal and Λ is chosen to produce the prescribed condition number $\kappa_2(A)$. Right-hand sides are sampled in FP64 from the standard normal distribution. An FP256 solve of the stored system provides the reference solution.

The generator also supports rotated-SPD and nonsymmetric random-SVD matrices, together with all-ones and random-sign solution modes.

11.3 Precision formats

The experiments use the following formats:

Table 11.2: Floating-point formats used in the experiments.

Format	Implementation	Significand bits	Unit roundoff
bfloat16	CPFloat<8, 8>	8	2^{-8}
FP16	CPFloat<11, 5>	11	2^{-11}
FP32	float	24	2^{-24}
FP64	double	53	2^{-53}
FP128	FP<64>	64	2^{-64}
FP256	FP<192>	192	2^{-192}

CPFloat simulates bfloat16 and FP16, while GMP provides the extended-precision formats. FP128 and FP256 are project labels for FP<64> and FP<192> and do not denote the corresponding IEEE formats. The main experiments use FP64 working precision. Each experiment specifies its factorization and residual precisions.

For FP64 working precision, $u^2 = 2^{-106}$, whereas the implemented FP128 format has $u_r = 2^{-64}$. The FP128-residual experiment therefore does not realize the commonly analyzed choice $u_r = u^2$, although it uses a residual precision higher than the working precision. The general forward-error bound remains applicable, but the attainment of $O(u)$ forward error is evaluated empirically.

11.4 Error measures

For each iterate, we compute

$$e_i = \frac{\|x_i - x_{\text{ref}}\|_{\infty}}{\|x_{\text{ref}}\|_{\infty}},$$
$$\eta_i = \frac{\|b - Ax_i\|_{\infty}}{\|A\|_{\infty}\|x_i\|_{\infty} + \|b\|_{\infty}}.$$

The forward error e_i and the normwise backward error η_i are both evaluated in FP256.

11.5 Termination criteria and statuses

Convergence is determined by the relative correction

$$\rho_i = \frac{\|d_i\|_2}{\|x_i\|_2}.$$

Refinement terminates with status `converged` when $\rho_i < u$. Otherwise, it returns `max_iterations` after 20 iterations, `factorization_input_non_finite` if A or b is non-finite after conversion to the factorization precision, `non-finite` if a non-finite value appears later, or `diverged` if the correction norm grows by more than a factor of 10 for three consecutive iterations. The iteration limit, growth factor, and required number of consecutive growth steps are configurable.

11.6 Experimental framework and reproducibility

Each run records its precision configuration, requested condition numbers, random seeds, termination status, and experiment-specific quantities. Fixed seeds and self-describing CSV files permit the experiments to be repeated and the figures to be regenerated independently.

12 Experimental design

We conducted five groups of experiments to examine how conditioning, precision choices, and residual handling affect the convergence and accuracy of mixed iterative refinement. Unless stated otherwise, they use the common setup described above and vary only the parameters relevant to their objective. Table 12.1 summarizes each group’s purpose, varied parameters, and recorded quantities.

Table 12.1: Overview of the five experimental groups.

Experimental group	Parameters varied	Quantities recorded	Purpose
Convergence histories	Representative $\kappa_2(A)$ values and precision triples	Per-iteration forward error, backward error, relative correction; final status	Examine behavior below, near, and above $\kappa_2(A)u_f = 1$.
Condition-number sweep	$\kappa_2(A)$ and precision triple	Final forward error, iteration count, final status	Determine how conditioning affects convergence and accuracy.
Residual-precision comparison	$\kappa_2(A)$ and residual precision	Final forward and backward errors, final status	Measure the benefit of higher residual precision.
Direct-solve comparison	$\kappa_2(A)$ and solution method	Final forward and backward errors, final status	Compare mixed IR with a direct working-precision solve.
Residual-scaling experiments	Residual scaling enabled or disabled	Error histories and conversion diagnostics	Determine whether scaling prevents residual information loss.

13 Results and discussion

13.1 Convergence histories

In this experimental group u_f , u_r and $\kappa(A)$ are varied; u and other common settings remain fixed. For each configuration, the forward error, backward error, and relative correction are recorded after every refinement step. Relative corrections are omitted from the final plots. These histories show the convergence rate, attained accuracy, and termination status.

For each factorization precision, the driver defines the reference condition number

$$\kappa_* = u_f^{-1}$$

and tests $\kappa_2(A) = 1$ together with representative values below, near, and above κ_* . Condition numbers are chosen as multiples of κ_* to provide a common relative scale across factorization formats. The condition $\kappa_2(A)u_f < 1$ serves as an approximate theoretical reference.

Curve labels include termination status. C, M, D, and F denote convergence, the iteration limit, detected divergence, and non-finite factorization input, respectively.

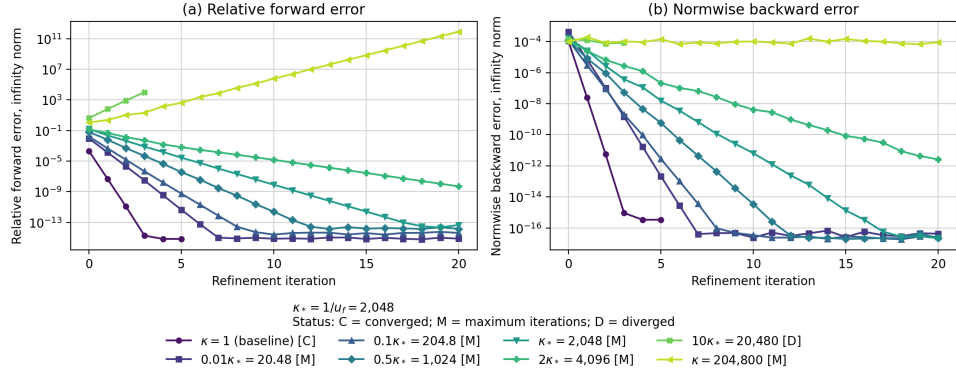
Four precision configurations are presented: FP16-FP64-FP64, FP16-FP64-FP128, FP32-FP64-FP64, and FP32-FP64-FP128. They form a 2×2 precision design with two factorization precisions and two residual precisions. This selection enables matched comparisons of how u_r affects the convergence history and attainable forward error at fixed factorization precision. It also shows how the convergence scale changes with u_f . The omitted FP8 and bfloat16 configurations examine alternative low-precision factorization formats and are summarized after the main comparison.

The convergence results are shown in Figure 13.1.

13.1.1 Interpretation

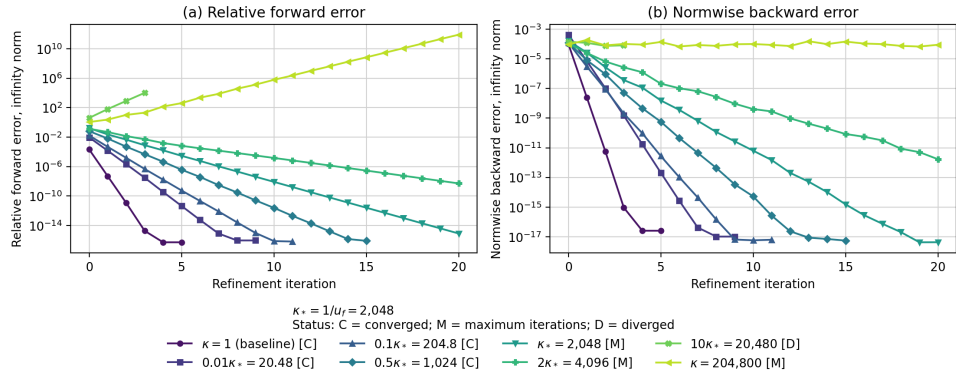
Across all four displayed triples, convergence slows as the condition number increases. Runs well below κ_* converge rapidly; runs near κ_* require more iterations, while runs sufficiently far above it reach the iteration limit or diverge. The transition occurs at different multiples of κ_* , consistent with $\kappa_* = u_f^{-1}$ being an approximate reference rather than a fixed convergence boundary.

Convergence histories: FP16-FP64-FP64 (n = 100, scaled)



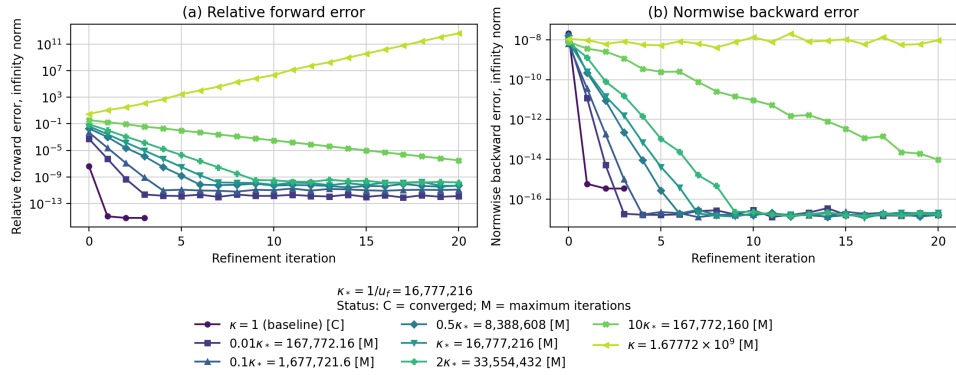
(a)

Convergence histories: FP16-FP64-FP128 (n = 100, scaled)



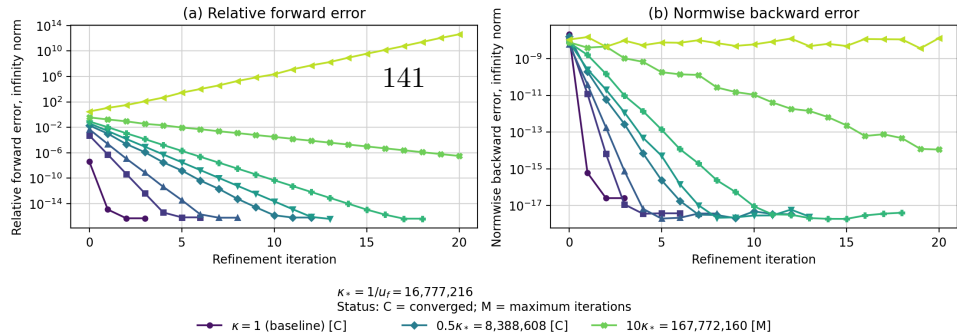
(b)

Convergence histories: FP32-FP64-FP64 (n = 100, scaled)



(c)

Convergence histories: FP32-FP64-FP128 (n = 100, scaled)



141

The 2×2 design separates the roles of the two precisions: u_f primarily determines the convergence rate and practical convergence region, whereas u_r primarily determines the attainable forward accuracy. The following subsections examine these effects and summarize the omitted FP8 and bfloat16 configurations.

Effect of residual precision

At each factorization precision, the FP64- and FP128-residual histories are initially nearly identical and separate only at the FP64 accuracy floor. Thus, u_r primarily controls attainable accuracy, not the initial convergence rate.

With FP64 residuals, $u_r = u$, and the forward error reaches a condition-dependent plateau. For FP16 factorization, runs from $0.01\kappa_*$ through κ_* stagnate between 10^{-15} and 10^{-14} and reach the iteration limit. For FP32, the larger absolute condition numbers produce clearer plateaus, rising from approximately 10^{-12} at $0.01\kappa_*$ to 10^{-10} at $2\kappa_*$. The backward errors still approach working-precision accuracy. This agrees with the expected forward-error limit $O(\text{cond}(A, x)u)$.

FP128 residuals reduce these plateaus. For FP16, runs through $0.5\kappa_*$ converge with forward errors of order 10^{-16} ; at κ_* , the error reaches approximately 8×10^{-16} without satisfying the stopping criterion within 20 iterations. For FP32, all runs through $2\kappa_*$ converge with errors between 4×10^{-17} and 7×10^{-17} . Backward errors are also lower, and more runs converge.

Once factorization limits convergence, both residual precisions produce nearly identical histories: from $2\kappa_*$ upward for FP16 and $10\kappa_*$ upward for FP32. Higher residual precision therefore improves stable runs but cannot prevent factorization-induced instability. Since HDNUM FP128 does not nominally satisfy $u_r = u^2$, these results demonstrate the benefit of $u_r < u$, but not the sufficient theoretical case $u_r = u^2$.

Factorization precision and convergence scale

At fixed residual precision, increasing factorization precision shifts convergence to larger absolute condition numbers. The reference values are

$$\kappa_*^{\text{FP16}} = 2^{11} = 2,048, \quad \kappa_*^{\text{FP32}} = 2^{24} = 16,777,216,$$

giving FP32 an 8,192 times larger convergence scale. At comparable κ/κ_* , both formats converge rapidly below κ_* , more slowly near it, and slowly or not at all sufficiently far above it. With FP128 residuals, FP16 converges through $0.5\kappa_*$ and FP32 through $2\kappa_*$. This is consistent with κ_* being an approximate reference rather than a fixed boundary.

Because each format uses its own normalized condition-number grid, the comparison involves different matrices and absolute condition numbers. It therefore shows convergence relative to each format’s scale, not the effect of changing u_f for the same system.

Alternative low-precision factorization formats

The bfloat16 and FP8 configurations were omitted to preserve the complete 2×2 FP16/FP32 design. They examine distinct low-precision formats and do not form a controlled pair.

Bfloat16 has lower significand precision than FP16 but a larger exponent range. It converges through $0.1\kappa_*$ but not from $0.5\kappa_*$ upward; divergence is detected at $10\kappa_*$. On the normalized grid, it converges more slowly than FP16. Its larger range provides no visible advantage here because residual scaling prevents underflow and no factorization-input range failure occurs.

FP8 is an extreme stress case. It converges at $\kappa = 1$ and $0.1\kappa_*$ in 10 and 17 steps, respectively, but reaches the iteration limit from $0.5\kappa_*$ through $10\kappa_*$. At $100\kappa_*$, conversion to FP8 produces non-finite input and factorization stops before refinement. These results expose the limitations of very low factorization precision but are secondary to the isolated u_f and u_r comparisons.

13.2 Condition-number sweep

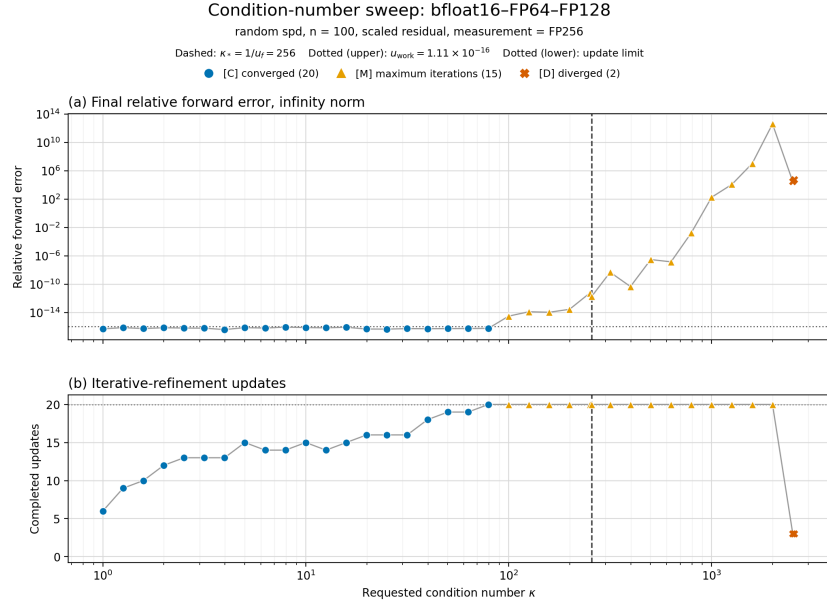
This experiment uses a dense logarithmic sweep to map final accuracy and iteration count over a broad condition-number range. For each factorization format, $\kappa_2(A)$ ranges from 1 to approximately $10\kappa_*$, where $\kappa_* = u_f^{-1}$. The working and residual precisions remain fixed at FP64 and FP128. The upper panels show the final relative forward error; the lower panels show completed refinement updates. Dashed lines mark κ_* , dotted lines mark u and the 20-update limit, and marker shapes indicate termination status.

Three configurations are shown: bfloat16-FP64-FP128, FP16-FP64-FP128, and FP32-FP64-FP128. Fixing u and u_r isolates the effect of factorization precision. The FP8 configuration is omitted as an extreme stress case and summarized below. The FP64-residual configurations are also omitted because their main effect was examined in the preceding experiment group.

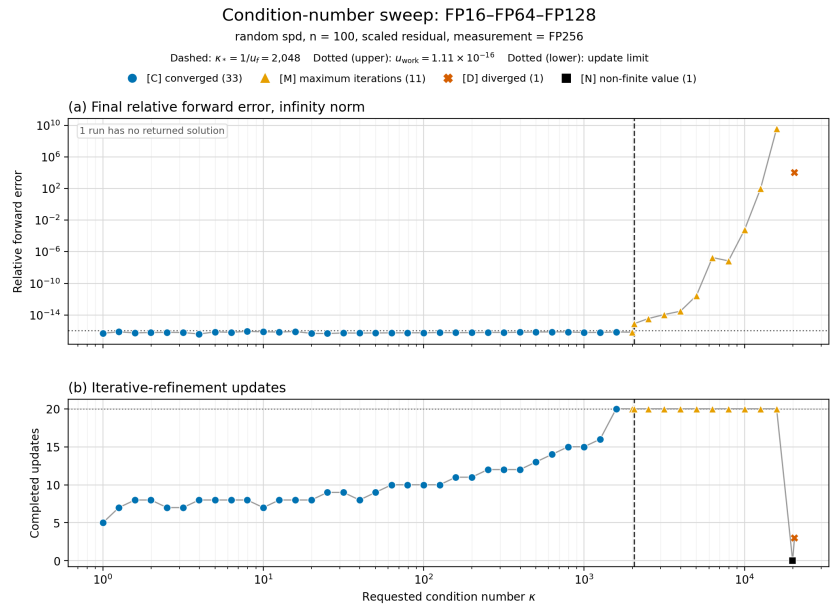
The results are shown in Figure 13.2.

13.2.1 Interpretation

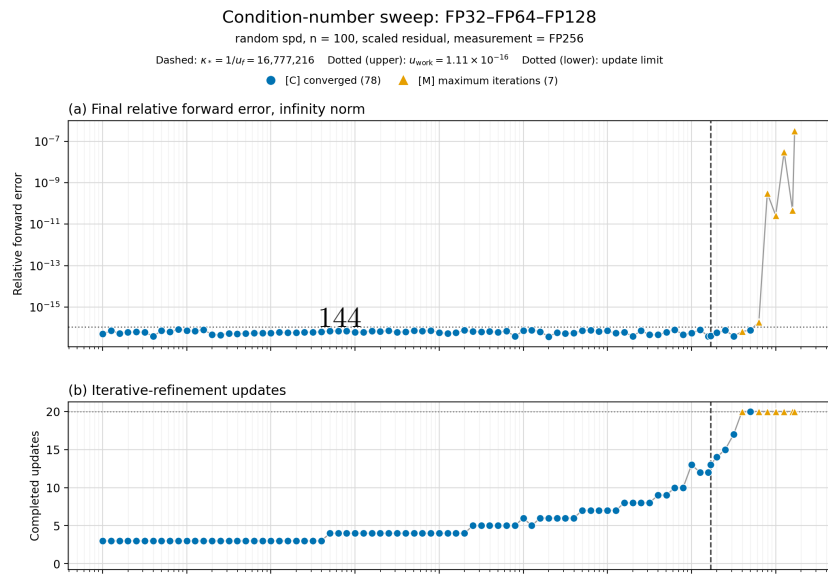
Across the three sweeps, increasing $\kappa_2(A)$ generally increases the required updates. The status totals are 20 converged, 15 maximum-iteration, and 2 diverged runs for bfloat16; 33 converged, 11 maximum-iteration, 1 diverged, and 1 non-finite run for FP16; and 78 converged and 7 maximum-iteration runs for FP32. Since the grids contain different numbers of points, these



(a)



(b)



144

totals describe each sweep but are not directly comparable success rates. Converged runs retain forward errors near working precision until the practical convergence limit is approached.

Conditioning and practical convergence region

Within each format, well-conditioned systems converge in few updates, while the update count rises toward the transition region. The first maximum-iteration runs occur at approximately $0.39\kappa_*$ for bfloat16, $0.97\kappa_*$ for FP16, and $2.37\kappa_*$ for FP32. The largest converged values are $0.31\kappa_*$, $0.77\kappa_*$, and $2.99\kappa_*$, respectively. Thus, higher factorization precision expands the practical convergence region on both absolute and normalized scales in these tests.

This trend is consistent with the theoretical convergence factor being of order $u_f\kappa(A)$, but $\kappa_* = u_f^{-1}$ is only an approximate scale. LU growth, factorization quality, componentwise conditioning, and norm-dependent constants also affect convergence. The normalized transition therefore need not occur at $\kappa/\kappa_* = 1$ or remain fixed across formats.

Final forward accuracy

Converged runs generally attain forward errors of order u . Increasing $\kappa_2(A)$ mainly raises the update count; final accuracy remains near working precision while refinement succeeds. The FP128 residual therefore avoids the condition-dependent floor observed with FP64 residuals.

Near the transition, some runs retain small errors but reach the 20-update limit because termination depends on the relative correction, not the forward error. Beyond this region, errors increase or become irregular. Final error must therefore be interpreted together with termination status.

Omitted configurations

FP8–FP64–FP128 is omitted as an extreme stress case: only 3 of 25 runs converge, up to approximately $0.10\kappa_*$. The FP16–FP64–FP64 and FP32–FP64–FP64 sweeps mainly reproduce the condition-dependent accuracy floor observed in the preceding experiment group. Only three runs in each sweep converge; most reach the 20-update limit because the correction criterion remains unsatisfied. The residual-precision experiment examines this effect directly.

13.3 Residual-precision comparison

This experiment isolates the effect of residual precision. The factorization and working precisions are fixed at FP32 and FP64, while the residual precision is varied between FP64 and

FP128. Both configurations use the same matrices, right-hand sides, condition-number grid, and algorithmic settings. Residual scaling is enabled, and errors are evaluated in FP256.

The upper panel shows the final normwise backward error; the lower panel shows the final relative forward error. The dashed line marks $\kappa_* = u_f^{-1}$, while the dotted lines mark u and the reference $\kappa_2(A)u$. Marker shapes indicate termination status.

The results are shown in Figure 13.3.

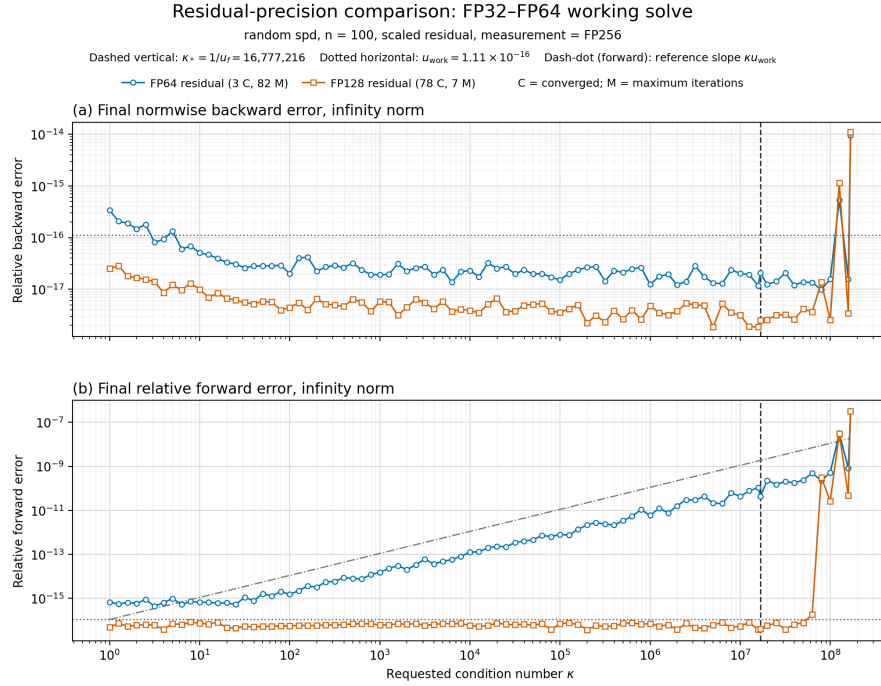


Figure 13.3

13.3.1 Interpretation

Across the stable range, both configurations attain backward errors of order u . Their forward errors separate as $\kappa_2(A)$ increases: with FP64 residuals, the error grows approximately in proportion to $\kappa_2(A)$, whereas with FP128 residuals it remains near working precision. Beyond the factorization convergence region, both variants become irregular.

Backward error

Over the stable range $10^2 \leq \kappa_2(A) \leq 10^7$, both backward-error curves are nearly independent of conditioning. FP64 residuals give errors between 1.2×10^{-17} and 4.2×10^{-17} , while FP128

residuals give approximately 1.9×10^{-18} to 6.7×10^{-18} . Both remain of order u .

FP128 residuals lower the observed values but do not change their theoretical order. This agrees with Carson and Higham (2018, Corollary 4.2), which predicts a limiting normwise backward error of order u , essentially independent of u_r . Values below u do not contradict this result, since the estimate specifies an order rather than a lower bound.

Forward error and residual precision

Over the stable range $10^2 \leq \kappa_2(A) \leq 10^7$, FP64 residuals produce forward errors from approximately 1.5×10^{-15} to 4.3×10^{-11} . Their log-log slope is 0.90, close to the predicted linear dependence on conditioning. With FP128 residuals, the error remains between approximately 3.8×10^{-17} and 8.1×10^{-17} , with no systematic dependence on $\kappa_2(A)$ and near working precision.

This agrees with the limiting bound

$$\frac{\|\hat{x} - x\|_\infty}{\|x\|_\infty} \lesssim u + 4pu_r \text{cond}(A, x)$$

of Carson and Higham (2018, Corollary 3.3). When $u_r = u$, the condition-dependent term gives an error of order $\text{cond}(A, x)u$. With sufficiently accurate residuals, this term becomes negligible and the limiting error is of order u . Since $\kappa_2(A)$ is used as a proxy for $\text{cond}(A, x)$, the plotted $\kappa_2(A)u$ line represents the expected scaling rather than an exact bound.

Only 3 of the 85 FP64-residual runs satisfy the stopping criterion; the remaining 82 reach the update limit. This mainly reflects the condition-dependent correction floor remaining above the u -based tolerance, even after the forward error reaches its expected level. With FP128 residuals, 78 runs converge and 7 reach the update limit. Beyond the FP32 factorization convergence region, however, both variants become irregular. Higher residual precision improves attainable accuracy but cannot compensate for an insufficiently accurate factorization.

13.4 Direct-solve comparison

This experiment compares mixed-precision iterative refinement with a direct solve. Iterative refinement uses FP32 factorization, FP64 working, and FP128 residual precision, with residual scaling enabled. The direct method performs a single LU factorization and solve in FP64. Both methods use the same matrices, right-hand sides, and condition-number grid. Errors are evaluated in FP256.

The upper panel shows the final relative forward error; the lower panel shows the final normwise backward error. The dashed line marks $\kappa_* = u_f^{-1}$, while the dotted lines mark u and the reference $\kappa_2(A)u_{\text{direct}}$. Marker shapes indicate the termination status of iterative refinement.

The results are shown in Figure 13.4.

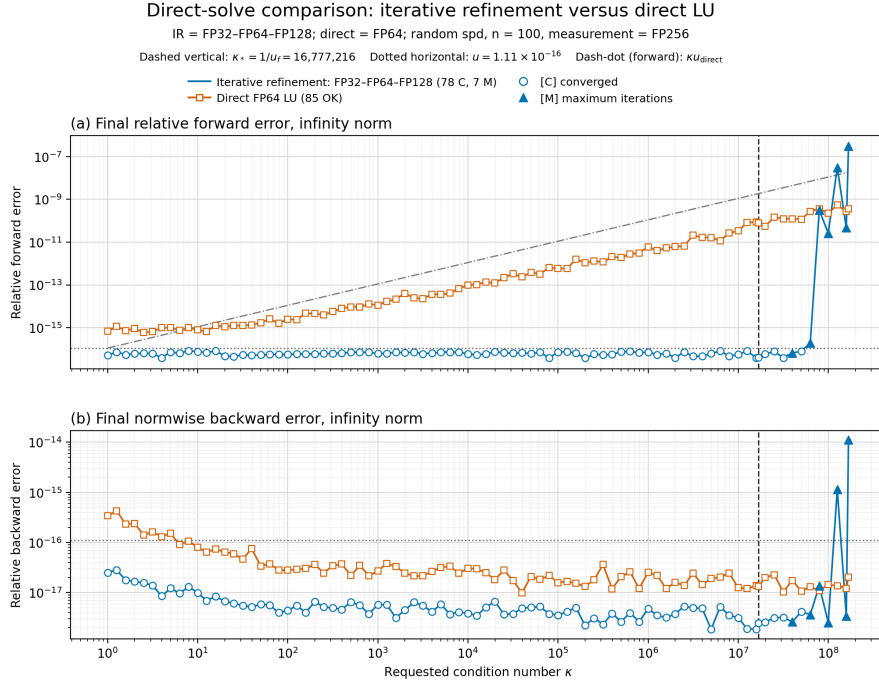


Figure 13.4

13.4.1 Interpretation

Within its convergence region, iterative refinement attains forward and backward errors near working precision. The direct FP64 solve remains backward stable across the full condition-number range, but its forward error increases with conditioning. Beyond the FP32 factorization limit, iterative refinement becomes irregular and loses its accuracy advantage.

Forward error and convergence region

Over the stable range $10^2 \leq \kappa_2(A) \leq 10^7$, the IR forward error remains near working precision, between 3.8×10^{-17} and 8.1×10^{-17} , with no systematic dependence on $\kappa_2(A)$. In contrast, the direct-LU error increases from 2.5×10^{-15} to 3.4×10^{-11} . Its fitted log-log slope is 0.82, broadly consistent with the expected $O(\kappa_2(A)u)$ behavior.

Of the 85 IR runs, 78 converge and 7 reach the maximum of 20 updates. The first maximum-iteration result occurs at $\kappa_2(A) \approx 3.98 \times 10^7$, while the largest converged result occurs at approximately 5.01×10^7 . The transition is therefore not a strict condition-number boundary.

Beyond this region, the IR errors become irregular and can exceed those of the direct solve. Accurate residuals and FP64 updates improve the solution only while the FP32 factors provide sufficiently accurate correction equations. The direct FP64 solve therefore becomes more reliable once the low-precision factorization limits IR convergence.

Backward error

Over the stable range $10^2 \leq \kappa_2(A) \leq 10^7$, both methods attain backward errors of order u , with little dependence on conditioning. The IR error lies between 1.9×10^{-18} and 6.7×10^{-18} , while the direct-LU error lies mostly between 1.0×10^{-17} and 3.8×10^{-17} . IR therefore gives consistently smaller observed errors, although both methods are backward stable.

The direct solve nevertheless exhibits condition-dependent forward error. A small backward error guarantees proximity to a nearby problem, but ill-conditioning can amplify this perturbation into a much larger forward error.

13.5 Low-precision limitations and residual scaling

For the FP16–FP64–FP128 configuration, the initial convergence histories showed forward and backward errors stagnating well above the expected FP64 accuracy. We attributed this to information loss when the shrinking FP128 residual was converted to FP16 before the correction solve.

The smallest positive normal and subnormal FP16 values are $2^{-14} \approx 6.10 \times 10^{-5}$ and $2^{-24} \approx 5.96 \times 10^{-8}$, respectively. Under round-to-nearest, magnitudes below approximately $2^{-25} \approx 2.98 \times 10^{-8}$ round to zero. Moreover, subnormal numbers have constant absolute spacing 2^{-24} and therefore increasingly poor relative resolution near zero. Residual conversion can consequently lose information both through rounding to zero and through severe quantization of surviving subnormal components.

To test this hypothesis, we instrumented the residual conversion at each iteration. The diagnostics record the residual infinity norm, its smallest nonzero component, the number of nonzero components before conversion, and how many become zero when converted to FP16. This allows conversion loss to be compared directly with the observed stagnation.

To reduce this loss, we normalize the residual before its conversion to FP16:

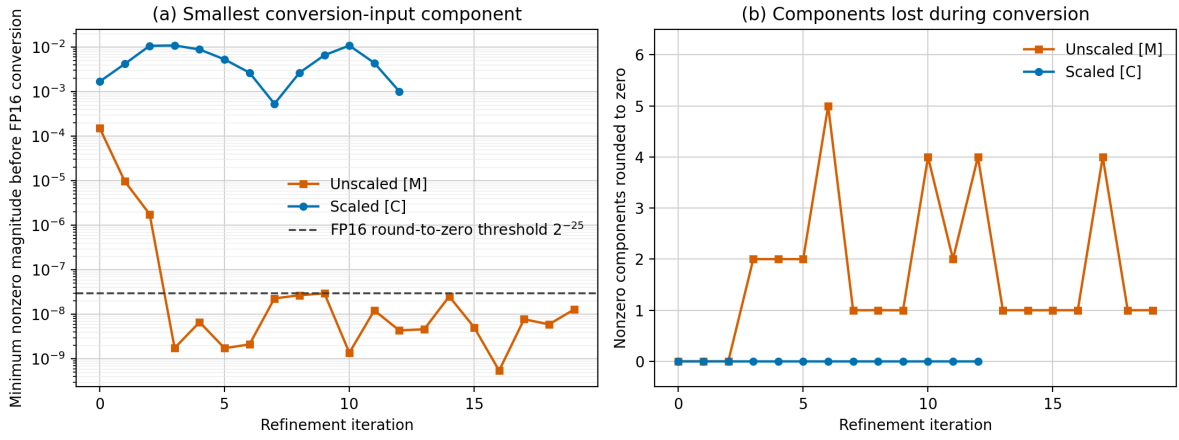
$$\theta_k = \|r_k\|_\infty, \quad \hat{r}_k = \frac{r_k}{\theta_k}, \quad A\hat{d}_k = \hat{r}_k, \quad d_k = \theta_k \hat{d}_k.$$

This is equivalent to the original correction equation in exact arithmetic, since $Ad_k = \theta_k A\hat{d}_k = \theta_k \hat{r}_k = r_k$. Scaling therefore leaves the mathematical correction unchanged while using the

FP16 range and resolution more effectively. The factor θ_k is reapplied after the low-precision solve in FP64.

The experiment compares scaled and unscaled refinement for $n = 100$ and $\kappa_2(A) = 500$, using the same matrix, right-hand side, and algorithmic settings. Residual scaling is the only varied option. In Figure 13.5, the conversion diagnostics show the residual magnitudes and components lost during conversion, while the convergence histories show the corresponding forward errors, backward errors, and relative corrections.

Residual-conversion diagnostics: FP16 / FP64 / FP128 ($n = 100$, $\kappa = 500$)



Effect of residual scaling: FP16 / FP64 / FP128 ($n = 100$, $\kappa = 500$)

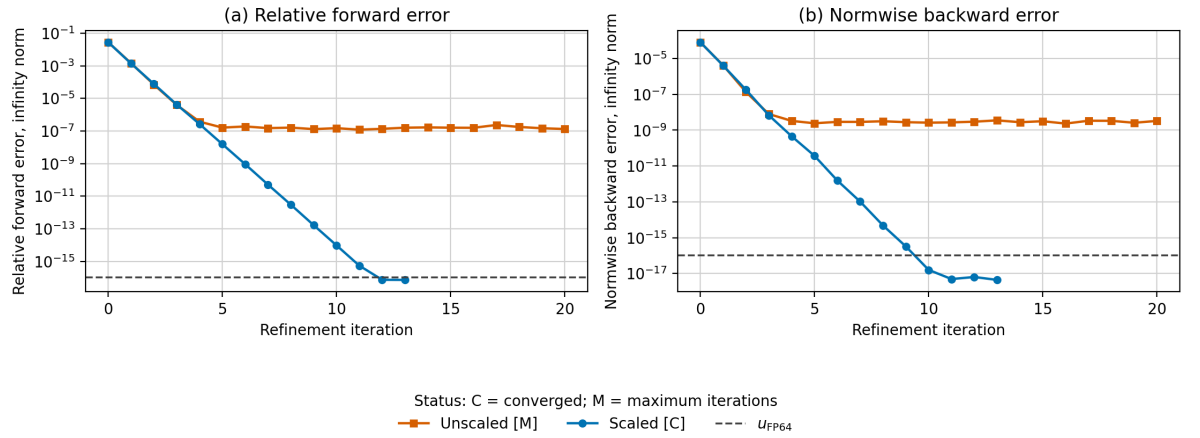


Figure 13.5:

13.5.1 Interpretation

The diagnostics confirm that information loss during unscaled FP16 residual conversion coincides with the onset of stagnation. Residual scaling eliminates the observed conversion loss, allowing the corrections and errors to continue decreasing until the method converges at FP64-level accuracy.

Residual-conversion diagnostics

In the unscaled run, the first conversion loss occurs at iteration 3, when the smallest residual component falls to 1.74×10^{-9} and two components round to zero. From iterations 3–19, all 100 components remain nonzero before conversion, but between one and five components per iteration become zero in FP16.

These counts measure only complete underflow to zero. They do not capture the additional information loss caused by distinct subnormal components being mapped to the same values on the coarse FP16 grid. The small number of zeroed components therefore understates the total effect of conversion.

With scaling, the smallest normalized components remain approximately between 5×10^{-4} and 10^{-2} , above the smallest FP16 normal value. Consequently, no component rounds to zero during any of the 13 updates, and the normalized residual avoids severe subnormal quantization.

Effect on convergence

Without scaling, the forward error stagnates between approximately 1.2×10^{-7} and 2.3×10^{-7} , while the backward error remains between 2.3×10^{-9} and 3.5×10^{-9} . The relative correction similarly plateaus near 2×10^{-7} , and the run reaches the maximum of 20 updates without converging.

With scaling, all three quantities continue decreasing. The method converges after 13 updates with final forward error 7.25×10^{-17} , backward error 4.44×10^{-18} , and relative correction 5.57×10^{-17} . Residual scaling therefore removes the observed stagnation and restores FP64-level accuracy.

This remedy specifically addresses underflow and quantization when the residual is converted to FP16. It does not prevent overflow when the matrix or right-hand side is converted to FP16, nor does it extend the convergence range imposed by the accuracy of the FP16 factorization.

13.6 Conclusion

The experiments broadly confirm the expected behavior of three-precision iterative refinement. Within the convergence region determined by the low-precision factorization, refinement attains FP64-level backward accuracy. Residual precision determines the attainable forward accuracy: with FP64 residuals, where $u_r = u$, the forward error grows with conditioning, whereas FP128 residuals keep it near working precision throughout the stable range.

Compared with a direct FP64 LU solve, iterative refinement produces substantially smaller forward errors as conditioning increases, but becomes unreliable once the FP32 or FP16 factors no longer provide accurate correction equations. The FP16 scaling experiments further show that limited exponent range can cause residual-conversion loss and stagnation; infinity-norm scaling prevents this loss and restores convergence without changing the mathematical correction.

The observed convergence in relatively few updates supports the potential of mixed-precision refinement to reduce computational cost on hardware with efficient low-precision arithmetic. Since the formats used here were partly software-emulated and execution times were not measured, performance gains remain to be evaluated experimentally on suitable hardware.

The experiments were implemented within a configurable framework that supports further studies with different precision combinations, test problems, condition-number ranges, and algorithmic options.

References

A GDB usage Notes

A.1 Basic GDB Commands

- **bt**: backtrace. Displays the active **call stack**, i.e. the chain of function calls that led to the program's current stopping point.

GDB lists the currently executing frame first as frame #0, then it's callers as #1 and the #nth callers caller as #n+1.

- **frame <n>**: switch to frame *n*.
- **info locals**: information about the stack variables.

Carson, Erin, and Nicholas J. Higham. 2018. "Accelerating the Solution of Linear Systems by Iterative Refinement in Three Precisions." *SIAM Journal on Scientific Computing* 40 (2): A817–47. <https://doi.org/10.1137/17M1140819>.

Higham, Nicholas J., and Theo Mary. 2022. "Mixed Precision Algorithms in Numerical Linear Algebra." *Acta Numerica* 31 (May): 347–414. <https://doi.org/10.1017/S0962492922000022>.